4章
データプロダクト管理

データプロダクトとはいったい何だろうと、疑問をお持ちかもしれません。単なるバズワードでしょうか。この章では、データプロダクトとは何なのかについて、分かりやすく説明します。大量のデータをほかのドメインに提供する際に必要不可欠な情報をすべて取り上げます。まず、この用語の曖昧さをなくすことから始めます。なぜなら、専門家によってさまざまな解釈や定義があるからです。次に、CQRS（Command Query Responsibility Segregation、コマンドクエリ責任分離）のパターンと、なぜそれがデータプロダクトアーキテクチャの設計に影響を及ぼすのかを考察します。データプロダクトのさまざまな設計原則について説明し、良く設計され、統合可能で、読み出し最適化されたデータモデルがコンシューマにとって重要である理由を学びます。その後、データプロダクトアーキテクチャについて見ていきます。データプロダクトアーキテクチャとは何か、どのように設計できるのか、一般的にどのような機能が必要なのか、そしてメタデータの役割などについてです。この章では、実例を使ってできるだけ具体的に説明します。この章が終わる頃には、データプロダクトアーキテクチャを活用することで、大量のデータをデータコンシューマが利用可能にする方法について、十分理解できるようになっていることでしょう。

4.1　データプロダクトとは何か

データ提供者に責任を負わせ、データ提供方法を非中央集権化することは、スケーラビリティを実現する上で非常に良い方法です。Dehghani氏は「data as a product（プロダクトとしてのデータ）」（https://oreil.ly/IsjlH）という言葉を使い、「データプロダクト」という概念を紹介しています。しかし、この2つには大きな違いがあります。**data as a product**は考え方を示しています。データオーナーとアプリケーションチームは、データを他者が管理するプロセスの副産物としてではなく、自分たちが責任を持つプロダクトとして扱わなければならないという考え方です。データプロバイダーはデータコンシューマをカ

スタマーとして扱い、カスタマーを満足させるような利用体験を提供しなければなりません。できる限り最高の体験を提供するために、データはどのように定義され、構築されるべきかを考えます。

データプロダクトはdata as a productという考え方とは異なり、アーキテクチャに対応するものです。データコミュニティ内では、データプロダクトがどのようにアーキテクチャと関係するかについて、さまざまな想定や解釈がなされています。一部の専門家は、データプロダクトは特定のコンテキスト境界に関連するデータを含んでいるだけの単なるデータセットではないと言います。データプロダクトには、データを収集し提供するために必要なすべてのコンポーネントや、メタデータ、データを変換するコードも含まれると言うのです。この解釈は、データプロダクトをアーキテクチャ量子（https://oreil.ly/yvZpx）と表現するDehghani氏の説明と一致します。アーキテクチャ量子とは、「メッシュ上のノードで、その機能に必要な3つのコンポーネントをカプセル化し、そのドメインの分析データへのアクセスをプロダクトとして提供するもの」のことです。Dehghani氏によれば、この3つのコンポーネントとは、コード、データとメタデータ、インフラストラクチャーのことです。つまり、Dehghani氏の主眼は明らかにソリューションアーキテクチャにあり、これにはデータの取得や、変換、保存、管理、共有に必要なすべてのコンポーネントが含まれます。

専門家の中にはデータプロダクトについて異なる見解を持っているものもいます。例えば、Accenture（https://oreil.ly/noV4B）はデータセット、分析モデル、ダッシュボードレポートをデータプロダクトと呼んでいます。このような見方は、データの物理的な表現に焦点を当てており、メタデータ、コード、インフラストラクチャーを必ずしも含まないため、Dehghani氏の見方とは大きく異なります。そのため、アーキテクチャを設計する前に、まずデータプロダクトという用語の意味を統一し、何が含まれるか、含まれないかを決める必要があります。

4.1.1　コード、データ、メタデータ、インフラストラクチャーを組み合わせる際の課題

理想的な世界では、データとメタデータはデータプロダクトとしてパッケージ化され、一緒に流通します。ING Tech Group Servicesのチーフインフォメーションアーキテクトである Ferd Scheepers氏は、Domain-Driven Design Europe 2022でメタデータ管理をテーマにプレゼンテーションを行いました。このプレゼンテーションの中で、彼は「メタデータはほかのデータに関する情

報を提供するデータである」ため、メタデータはデータ管理にとって非常に重要であると説明しました。その重要性を明確にするために、彼はオランダで花がどのように流通しているかという例を用いて説明しています。

アールスメール花市場は、世界最大のフラワーオークションです。月曜日から金曜日まで、毎日約4,300万本の花と500万本の植物が売買されています。オークションには、毎日世界中から業者が参加します。到着した花はラベルが貼られ、指定された場所に送られます。バイヤーにとっては、すべてのプロセスがシームレスです。デジタルで注文し、数量を送信し、価格と送り先を指定するだけです。このオークションの素晴らしいところは、完全なメタデータ駆動型であることです。すべてのフラワーボックスにはバーコードが貼られ、原産国や都市、数量、入札開始価格、重量、生産日と消費期限、生産者名など、花に関する重要な情報が含まれています。花が落札されると、別のバーコードが追加され、そこには出荷のための情報、つまり購入者のオーナーシップの詳細、目的地、出荷指示などが含まれます。メタデータがなければ、花のオークションは成り立ちません。

花のオークションでは、花とメタデータは常に一緒に管理・流通されます。物理的なオブジェクトで、互いにつながっているからです。しかし、デジタルの世界ではそううまくはいきません。このことをはっきりさせるために、いくつか例を挙げてみましょう。全データを管理し説明するために中央データカタログを使う場合、データとメタデータは別々に管理されます。この例では、メタデータはすべてデータカタログにあり、物理データはすべてほかの場所に保存されています。例えば、データの抽出とロードのプロセスを管理するようなメタデータ駆動型のインジェストフレームワークでも、同じように分離されます。このようなフレームワークでは、データ処理を開始する前に何百ものデータプロダクトコンテナからメタデータをクロールすることはしません。メタデータ駆動型のインジェストフレームワークは、アーキテクチャのコンポーネントとしてデータ自体から切り離され、独立したデータベースでメタデータを管理するのが一般的です。このメタデータを利用して、データを抽出、結合、変換、ロードするプロセスと依存関係をコントロールします。

リネージやデータ品質情報も同様です。関連するメタデータをデータプロダクトコンテナ自体にまとめてしまうと、リネージやデータ品質をエンドツーエンドで監視するために、すべてのデータプロダクトにわたって複雑な連合型のクエリを実行する必要があります。あるいは、すべてのメタデータを中央の別の場所に複製する方法も考えられますが、そうするとアーキテクチャ全体が大幅

に複雑になってしまいます。また、物理的な表現が複数存在する同じデータについて、セキュリティ、プライバシー分類、機密ラベル、オーナーシップ情報をどのように扱えば良いのでしょうか。メタデータとデータを密結合にすると、データプロダクトをコピーするとメタデータが複製されてしまいます。両者は同じアーキテクチャに存在するためです。メタデータの更新を行う場合、複数のデータプロダクトコンテナに対して同時に行わなければなりません。これは複雑な作業です。常にすべてのコンテナが利用可能でアクセス可能でなければならないからです。オーバーレイアーキテクチャを使用してこれらの更新を非同期に実行することもできますが、この場合もアーキテクチャ全体が著しく複雑になります。

結論をまとめると、データプロダクトを、コード、データ、メタデータ、インフラストラクチャーを一緒にまとめたコンテナと見なすのはあまりにも単純すぎます。データプロダクトの定義については、近年のデータプラットフォームがどのように機能しているかという実態を踏まえて、定義を改める必要があります。

4.1.2　論理エンティティとしてのデータプロダクト

これまでの定義が適切でないとすれば、データプロダクトはどのように定義されるべきなのでしょうか。筆者は、データとテクノロジーの管理は、2つの異なるアーキテクチャ視点から行うべきだと考えています。1つはデータ管理の問題に対応するものであり、もう1つは基盤となるテクノロジーアーキテクチャの管理の問題に対応するものです。図4-1に、このような考え方を示します。

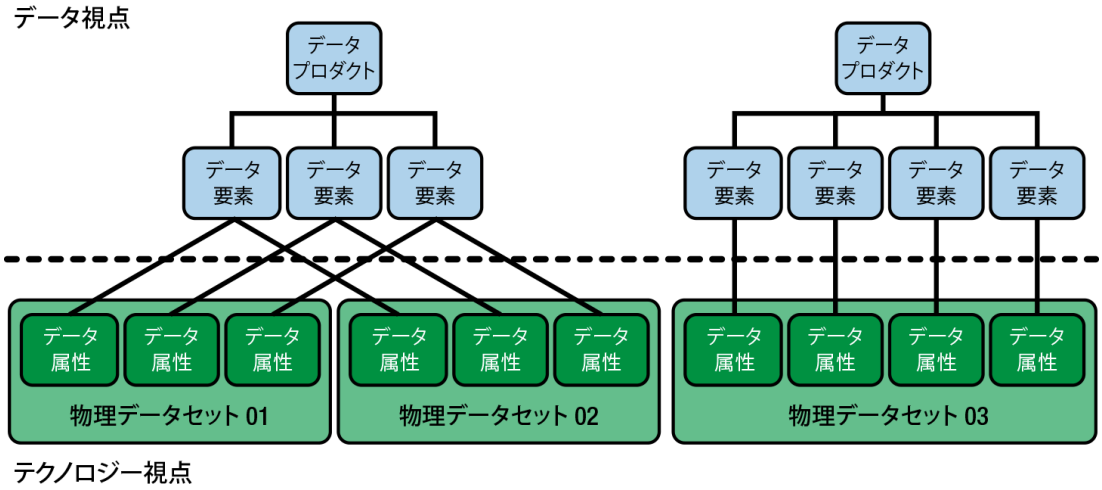

図4-1　データとテクノロジーを管理するための別々の視点

なぜ、データ管理とテクノロジー管理を分けるのでしょうか。まず、アーキテクチャをよりコスト効率の高いものにし、インフラストラクチャー管理における複雑さのオーバーヘッドを減らすことができるからです。複数の「データプロダクト」を管理する「データプロダクトアーキテクチャ」を使うという選択肢ができます。

次に、データとアーキテクチャを分離することで、（意味の）同じデータを流通させなければならない場合にうまく処理することができます。例えば、クロスドメインのデータ検証やリッチ化を行う場合、データを流通しなければならなくなることがあります。また、複数のユースケースを同時に処理するために、データの複製や前処理が必要になる場合があります。例えば、全く同じデータをParquetとDelta Lakeのファイル形式で保存する場合などです。この場合、データの基本的な意味づけを変更することなく、データを複製することになります。このような例を**図4-1**に示しています。物理データセット01と02は同じ意味を持つので、それぞれの物理データ属性は同じ要素にリンクしています。

そして3つ目は、データとメタデータの密結合を避けられることです。データとメタデータが同じコンテナに常に一緒に格納されている状況を想像してみてください。例えば、オーナー、ビジネスエンティティ、分類が変更された場合、データプロダクトアーキテクチャ内に存在する、そのデータに対応するメタデータについても、すべて変更しなければなりません。さらに、データオーナーシップの一貫性もどのように担保するのでしょうか。メタデータを複製すれば、データオーナーシップも変わってしまうリスクがあります。同じ意味のデータであれば、異なる場所にある異なる物理的表現を持つからといって、このようなことがあってはなりません。

データとメタデータを分離することで、データプロダクトの定義が変わります。**データプロダクトとは、消費されることを目的とする、データを表す自律的な論理エンティティのことです**。この論理エンティティから、基盤となるテクノロジーアーキテクチャ、つまり、読み出しに最適化され、コンシューマがすぐに利用できる物理的なデータ資産が保存されている物理的な場所との関係が構築されます。データプロダクト自体には、論理データセット名、出所ドメインとの関係、一意なデータ要素、ビジネス用語に関する記述、データセットのオーナー、物理データ資産（実際のデータそのもの）への参照などの情報が含まれます。ビジネス上では意味的に一貫したものですが、物理的なレベルでは複数の異なる形式や表現を持つことがあります。このように形式は変化する

ことがあるため、データプロダクトはテクノロジーに依存しないように定義する必要があります。メタデータは抽象的なままにしておき柔軟性を残します。

9章では、データプロダクト管理のためのメタモデルについてスクリーンショットを示し、データプロダクトの論理的な視点を具体的にしていきます。この仕組みを早く学びたいという方は、「9.2　エンタープライズメタデータモデル」を読んでから、ここに戻ってこられることをお勧めします。

データプロダクトを論理エンティティとして定義すると、具体的な実装を意識することなく、データ（ドメイン、データオーナー、組織メタデータ、プロセス情報、そのほかのメタデータなど）を記述、分類、ラベル付け、リンクすることができます。データプロダクトの1つ下のレベルに、データ要素があります。データ要素は、情報の原子単位です。物理データ、インタフェースメタデータ、アプリケーションメタデータ、データモデリングメタデータへの接続端子として機能します。物理データを記述するには、スキーマ情報、データタイプなどの技術的なメタデータが必要です。

データプロダクトがどのようなものかを理解したところで、その設計とアーキテクチャにどのような影響があるのかを考察していきましょう。まずCQRSから始め、読み出し最適化モデルなど設計の原則について見ていきます。その後、少し視野を広げて、データプロダクトアーキテクチャを検討します。

4.2　データプロダクトのデザインパターン

データプロダクトの定義を変えたからといって、プロダクトとしてデータを管理するという概念を捨てなければならないわけではありません。アプリケーションオーナーやアプリケーションチームは、データをほかの人が管理するプロセスの副産物ではなく、自分たちが責任を持つプロダクトとして扱わなければなりません。データプロダクトはデータコンシューマのために作られるものです。データプロダクトには、定義された形式、インタフェース、メンテナンスと更新のサイクルがあり、それらはすべてドキュメント化されています。デー

タプロダクトには、処理されたドメインデータが含まれます。このデータは、サービスレベル目標（SLO、Service Level Objective）のインタフェースを介して下流のプロセスと共有されます。特別な要件がない限り、（技術的な）アプリケーションデータは、合意された品質基準を満たすように処理、整形、クレンジング、集約、（非）正規化された後にはじめて消費できるようになります。

1章では、データ転送の制約、トランザクションシステムの設計、データ消費量の激増への対応など、エンジニアリング上の問題について学びました。忙しく稼働している運用システムから大量のデータを取り出すことは、負荷が高すぎてシステムがクラッシュしたり、制御不能になったり、利用できなくなったり、さらに悪いことに破損したりする可能性があるため、危険です。そのため、読み出し専用かつ読み出し最適化されたデータを作成し、このデータを消費できるようにするのが得策です。このための一般的なデザインパターンであるCQRSについて見てみましょう。

4.2.1　CQRSとは何か

CQRS（コマンドクエリ責任分離）（https://oreil.ly/qmeLE）は、大量読み出し用にデータのコピーを作成することを基本とする、アプリケーションデザインパターンです。

操作コマンドと分析クエリ（それぞれ、**書き込み**と**読み出し**と呼ばれます）は全く異なる操作です。CQRSパターンでは、これを別々に扱います（**図4-2**を参照）。ビジー状態のシステムの負荷を調べると、コマンド側が計算リソースの大半を使用していることがわかります。これは当然のことです。なぜなら、書き込み、更新、削除の操作を正常に実行するためには、データベースは通常、次に示すような一連のステップを実行する必要があるからです。

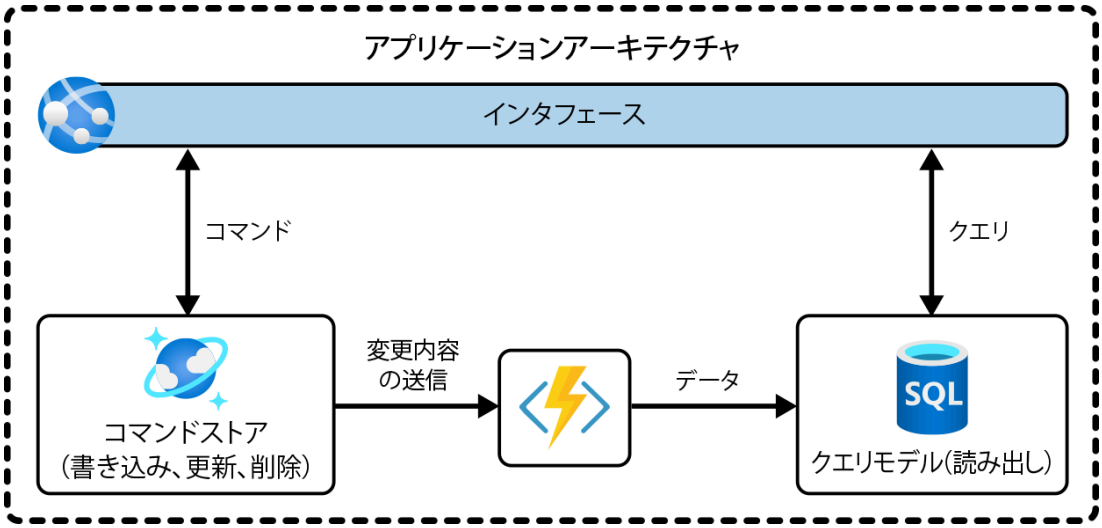

図4-2　CQRSを使用するアプリケーションは、トランザクション用のコマンドモデルと読み出し用のクエリモデルという、2つの異なるデータモデルを使用することで、クエリとコマンドを分離する

1. 利用可能なストレージの量を確認します。
2. 書き込み用にストレージを割り当てます。
3. テーブル/カラムのメタデータ（型、制約、デフォルト値など）を取得します。
4. レコードを検索します（更新や削除の場合）。
5. テーブルまたはレコードをロックします。
6. 新しいレコードを書き込みます。
7. 挿入された値を検証します（一意、正しい型など）。
8. コミットを実行します。
9. ロックを解除します。
10. インデックスを更新します。

データベースの読み出しは、書き込みに比べて、実行しなければならないタスクが少ないため、計算リソースが少なくて済みます。最適化のため、CQRSは書き込み（コマンド）と読み出し（クエリ）を、この図に示すように2つのモデルを使って分離します。一度分離した後は、両者を同期しなければなりませんが、これは通常、変更内容を含むイベントを発行することによって行われます。図4-2では「イベント矢印」（稲妻のアイコン）で示されています。

CQRSのメリットは、書き込みと読み出しに同じ種類のデータベースを使う必要がないことです[†1]。書き込みデータベースは目的に応じて複雑なものを使用していても、読み出しデータベースを読み出し性能を基準に最適化することができます。さまざまなユースケースに対して異なる要件がある場合、複数の

読み出しデータベースを作成し、それぞれが特定のユースケースに最適化された読み出しモデルを実装することもできます。要件が異なれば、書き込みデータストアは同じでも、読み出しストア間で一貫性モデルを使い分けることもできます。

[†1] CQRSの欠点は、複雑になってしまうことです。2つのモデルを同期させるためにレイヤーを増やす必要があります。また、遅延も発生するおそれがあります。

CQRSのもう一つのメリットは、読み出しと書き込みの両方を同時に大規模化する必要がないことです。リソースが足りなくなったら、どちらか一方だけを大規模化することができます。書き込みと読み出しの両方に、単一のモデルで対応する必要がないことで、技術面での自由度という最大のメリットも享受できます。読み出しを非常に高速に実行する必要がある場合や、異なるデータ構造が必要な場合は、コマンドストア用にはACIDデータベース（原子性、一貫性、独立性、永続性）を使いつづけながら、読み出しストアには別のテクノロジーを選択することができます。

CQRSはソフトウェアエンジニアリングのデザインパターンであり、ある特定の（そして恐らくより大規模な）システムの設計プロセスを改良する上で役立ちます。そして、データプロダクトアーキテクチャの設計とビジョンに大きな影響を与えるはずです。**アーキテクチャの将来的なモデルでは、ほかのアプリケーションがデータを大量に読み出したいときは、アプリケーションごとに少なくとも1つの読み出しデータストアが作成されたり、使用されたりしなければなりません。**このアプローチは、将来のアーキテクチャの設計と実装をシンプルにします。また、データを大量に消費する場合のスケーラビリティも高まります。

CQRSは**マテリアライズドビュー**と深く関係しています[†2]。データベース内のマテリアライズドビューは、元のテーブルによって更新されつづけるデータの物理的なコピーです。マテリアライズドビューはパフォーマンス向上のためによく使われます。元

のテーブルからデータをクエリする代わりに、事前に計算され最適化された、マテリアライズドビュー内のサブセットに対してクエリが実行されます。このように、元となるテーブル（書き込みに使用）とマテリアライズドサブセット（読み出しに使用）を分離することは、CQRS内で見られるものと同じです。ただし、両方のデータコレクションが2つの異なるデータベースではなく、同じデータベースに存在するという微妙な違いがあります。

[†2] TechDifferences（https://oreil.ly/PAoPD）に、データベースビューとマテリアライズドビューの違いについての概要が掲載されています。

4.2.2　データプロダクトとしての読み出しレプリカ

データを読み出すソースとしてレプリカを使うことは今に始まったことではありません。実際、レプリカ、コピー、データウェアハウス、データレイクからの読み出しはすべて、ある種のCQRSと見なすことができます。一般的にトランザクション指向のアプリケーションを実行・管理するオンライントランザクション処理（OLTP）システムと、運用レポート作成に使われるオペレーショナルデータストア（ODS）の間で、コマンドとクエリを分割するのも同じようなものです。この例では、ODSはOLTPシステムのデータのレプリカです。これらのパターンはすべて、操作用のデータベースから読み出しデータベースを構築するというCQRSの哲学に沿ったものです。

Martin Kleppmann氏は、「turning the database inside-out（データベースの裏返し）」パターン（https://oreil.ly/YToL6）を使っています。このパターンもCQRSの1つですが、イベントストリーミングによる事実の収集を重視しています。このパターンについては6章で説明します。

データプロダクトアーキテクチャもCQRSの哲学に従うべきです。データプロダクトはデータの複製という位置づけであり、データコンシューマが大量のデータを読み出すために使われます。データプロダクトは高いレベルで管理され、ドメインやアプリケーションからそのコンテキストを継承します。簡単に言えば、**図4-3**に示すように、データプロバイダーとデータコンシューマがデータを交換したい場合は、必ずデータプロダクトアーキテクチャを使用しなければならないということです。

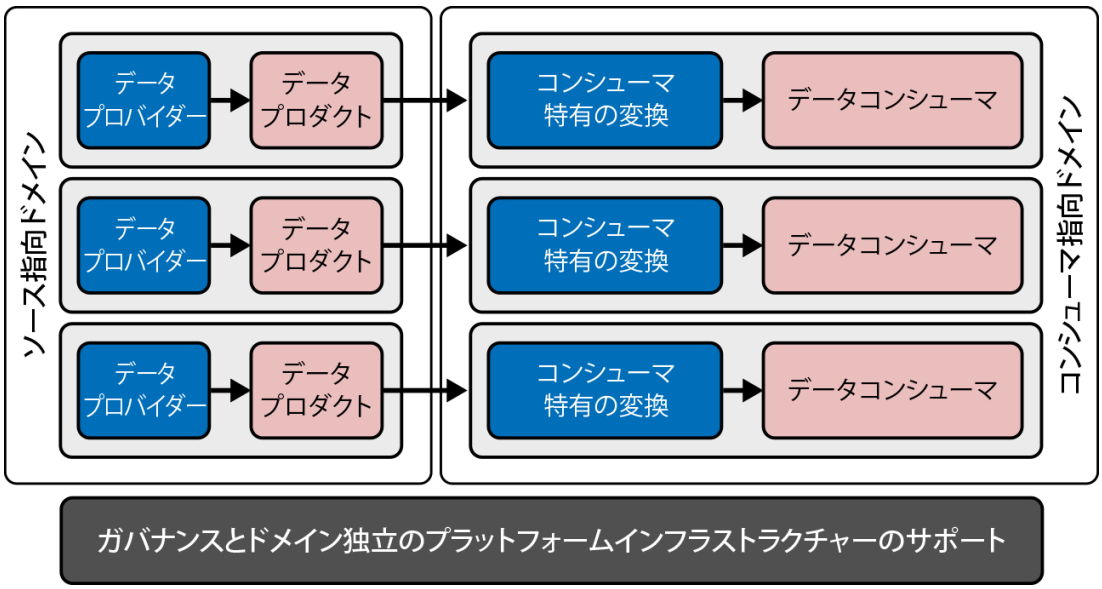

図4-3　データプロバイダーとデータコンシューマの非中央集権型コラボレーション

データプロダクトは左側のデータプロバイダーの近くに、変換ステップはデータコンシューマの近くに配置されていることに注目してください。この配置は、コンシューマにデータを提供する際の手法を統一したためです。単一の統合データモデルを使用するのではなく、設計を変更し、すべてのコンシューマアプリケーションに対して、ドメインデータをクレンジングし、容易に消費す

ることができるようにしたものを提供します。このデータは振る舞いや機能を提供するものではありません。そのため、運用データとは性質が異なります。誰もが読み出しやすいように最適化されているので、データをすばやく価値に変えられるようになっています。

4.3　データプロダクトの設計原則

読み出しに適した、ユーザーフレンドリーなデータを作成することが重要です。これは簡単なことのように思えますが、実際には想像以上に大変な場合が多いのです。具体的には、データをどのように取得し、構造化し、モデル化するかを検討する必要があります。難しいのは、多くのチームと共同でデータプロダクトを何度も作り上げる必要がある点です。これを効率的に行い、データコンシューマが増えるたびに新たにデータプロダクトを開発したり、複雑なデータ構造の検討やデータ品質の問題を解消するために長期的なサイクルを繰り返したりするような事態は避けなければなりません。再利用性をできるだけ確保して、データを簡単に処理したいと思っているはずです。この節では、データプロダクトを設計する際に役立つ設計原則を紹介します。

4.3.1　リソース指向の読み出し最適化設計

継続的に再学習する分析モデルは、絶えず大量のデータを読み出しています。これはデータプロダクトの設計に影響を及ぼします。なぜなら、データの読み出しやすさを最適化する必要があるからです。ベストプラクティスは、APIを設計する際にリソース指向を意識することです。リソース指向のAPIは一般的にリソース階層としてモデル化され、各ノードは単純なリソースかコレクションリソースのいずれかになります。便宜的に、これらはそれぞれリソース、コレクションと呼ばれることが多いです。

データプロダクトの場合、データを論理的にグループ化し、それぞれのデータセットが同じ種類のデータの集まりを表すように、対象分野ごとにクラスタリングします。こうすることで、同じリソース指向のアプローチを採用することができます。このような設計では、データプロダクトのデータはあらかじめ計算され、非正規化され、マテリアライズされます。消費側では、計算コストの高いデータ結合を実行する必要がないため、下流での消費がよりシンプルで高速になります。また、一緒に存在するべきデータが論理的にグループ化されているため、適切なデータを見つけるのに要する時間も短縮されます。

データプロダクトにリソース志向設計を採用する場合、過度に正規化されたり技術的すぎたりする物理データモデルを、より再利用しやすく論理的に整列されたデータセットに変換する必要があります。これにより、Kimball氏のスタースキーマやInmon氏のデータマートのように、非正規化され読み出しに最適化されたデータモデルになります。また、これは複雑なアプリケーションロジックを抽象化しなければならない、ということでもあります。データは、できるだけ多くのコンシューマを満足させるために、適切な粒度でほかのドメインに提供されなければなりません。リンクされたデータプロダクトの場合、相互参照と外部キーの関係がデータプロダクトのセット全体を通して一貫していなければなりません。例えば消費ドメインは、さまざまなデータセットを結合するためにキーを操作する必要はないはずです。このアプローチでデータプロダクトを構築するということは、ドメインが意味的なロジックを保有し、読みやすさを向上させるためにデータをどのように変換するかについて責任を持つことを意味します。この話はこのあたりで終えて、ほかのベストプラクティスと設計原則を見てみましょう。

データプロダクトとデータボルト

データボルトモデルを使ってデータプロダクトを設計できるかどうか、気になっているかもしれません。データボルトは、データウェアハウスの構築のために使われる、データのモデル化に関するデザインパターンです[†3]。データボルトは、関連付けを持つエンティティを分離することで、柔軟性、保守性、スケーラビリティを実現することを狙いとしています。かなり複雑な構造であるため、この技術の訓練を受けていない「門外漢」にはデータを理解するのが難しくなります。さらに、きめ細かなデータモデリングの手法は、大規模クラウドベンダーの分散ストレージアーキテクチャと合わないことが多いため、アーキテクチャの処理速度が遅くなり、コストも高くなります。多くのコンシューマがデータの結合と統合を繰り返す必要があります。

図4-4は、データボルトスキーマと単純なスタースキーマを比較したものです。簡単に言えば、データボルトの方法論では、エンティティをハブ、リンク、サテライトという言葉に置き換えて継続的に進化するビジネス要件をサポートします。

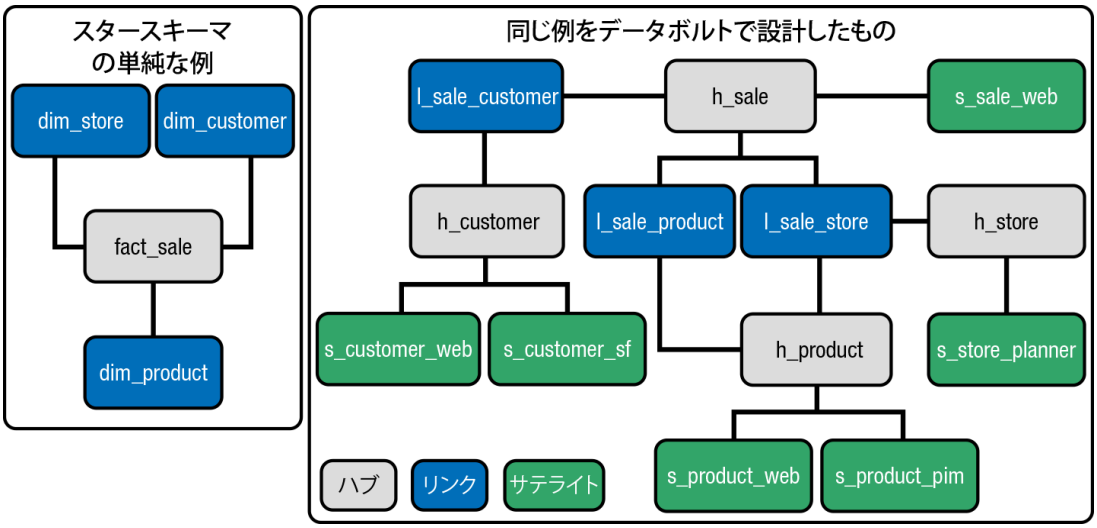

図4-4　データボルトの手法と単純なスタースキーマとの比較（出典：PhData、https://oreil.ly/lhf6m）

[†3] Morrisのデータボルト2.0に関する記事（https://oreil.ly/XxYgg）では、このデザインパターンについて「the good, the bad and the downright confusing（良い点、悪い点、そして実に分かりにくい点）」が述べられています。

4.3.2　データプロダクトのデータは不変

データプロダクトのデータは**不変**であり、したがって**読み出し専用**です。なぜ読み出し（専用）データストアは不変でなければならないのでしょうか。不変であることで、同じデータを何度でも繰り返し再生成できることが保証されます。読み出し専用の設計を使用すれば、同じドメインデータに対して異なるバージョンの真実が存在するということはありえません。この原則に従ったデータプロダクトアーキテクチャでは、新しい意味のデータは作成されません。真実はゴールデンソースアプリケーションでしか変更できません。

4.3.3　ユビキタス言語の使用

データがどのように作られたのか、そのコンテキストを理解することが重要です。そこで、ユビキタス言語の出番となります。ユビキタス言語とは、ステークホルダーと設計者が取り決めた、設計のニーズに応えるために構築され形式化された言語のことです。ユビキタス言語はドメイン、つまりビジネス機能と目標に沿ったものでなければなりません。ドメインのコンテキストがデータプ

ロダクトの設計を決定します。一般的な実装としては、ドメインからの情報とともに、これらの情報がすべて格納されたデータカタログを用意することです。コンテキストを公開し、ドメインに対して定義を明確にすることで、誰もがデータの出所と意味を確認し、それに基づいた合意をすることができます。

組織によっては、データプロダクトの物理データセットにおいて、人間にとってわかりやすいカラム名を使用することをドメインに対して義務付けているところもあります。物理データモデルから概念データモデルへのマッピングが示されている限り、これは必ずしも必要なことではありません。このマッピングにより、ドメインはビジネスコンセプトをユビキタス言語から物理データに変換することができます。

4.3.4　ソースから直接取り込む

筆者は以前の仕事で、企業がシステムからシステムへとデータをやり取りする際に、データチェーンが長くなることが多いことを学びました。データ管理にとって、これは問題になります。データの出所がはっきりとしないからです。そのため、常にデータを元のシステムから取得するようにしてください。ゴールデンソースから一意のデータを使用することで、データアクセスとデータ管理の両面で真実のソースが1つであることが担保されます。これは、データプロダクトが区別され明示的であることを意味しています。つまり、データプロダクトは単一のドメインに属し、隔離され、ほかのドメインのアプリケーションと直接的な依存関係を持つことはできません。この設計原則はまた、ドメインに対して、データオーナーの異なるほかのドメインのデータをカプセル化することを禁じる、という意味でもあります。

4.3.5　相互運用標準の明確化

データプロダクトが、どのようにほかのドメインに提供されるかを定義することが重要です。データプロダクトは、データベース、標準化されたファイルフォーマット、グラフ、APIエンドポイント、イベントストリームなどの形式で提供されます。どのような場合でも、インタフェースの仕様を標準化すべきです。もしすべてのドメインが独自の標準を定義してしまうと、データ消費は非常にやりにくくなります。このため、標準化は極めて重要です。そこで、さまざまな流通形式（バッチ型、API型、イベント型など）や、メタデータの標準などをはっきりと定義する必要があります。データプロダクトのアーキテクチャがどうなっているかを具体的に示してください。例えば、バッチデータプロ

ダクトはDeltaフォーマットで公開され、データカタログに登録されなければならないなどです。

4.3.6　生データは使わない

データプロダクトは生データとは対極にあるものです。生データを公開するには、消費するすべてのドメインで再加工が必要になるからです。したがって、どのような場合でも、レガシーシステムや複雑なシステムをカプセル化し、技術的なアプリケーションデータを隠すべきです。例えばリサーチ分析など、どうしても生データを利用したい場合は、フラグを立て、デフォルトでは一時的にしか利用できないようにしましょう。生データ（変換されていないデータ）には何の保証もありません。

システムは生データを公開するインタフェースを作っていいのか

アプリケーションが生データと消費最適化されたデータを同時に提供できるようにすることを検討するとよいでしょう。このアプローチは、データサイエンティストと、すぐにアクセスしたいコンシューマの両方にメリットがあります。生データを提供するインタフェースは、プライベートと明記し、無保証とすべきです。生データを提供するインタフェースを本番データパイプラインとしては絶対に使用すべきではありません。

2章で説明した「生データを使用しない」という原則は、外部データプロバイダー（企業のエコシステムの論理的境界外で事業を行う外部パーティ）にも適用されます。これらのデータプロバイダーは、一般的に、管理外の別のネットワーク上の場所で業務を行っています。エコシステムのパートナーには、標準に準拠するよう要求するか、中間ドメイン（複雑さを抽象化し、安定かつ安全な消費を保証する仲介役として機能する消費ドメイン）を介してメディエーション（調停）を行います。

4.3.7　コンシューマに合わせない

同じデータプロダクトが、さまざまなユースケースのために、さまざまなドメインチームによって何度も利用されることが考えられます。そのため、チームはデータプロダクトを（個々の）データコンシューマの特定のニーズに合わせるべきではありません。ドメインの生産性を最大化し、消費と再利用を推進することが第一の設計原則です。

一方、データプロダクトはユーザーからのフィードバックに基づいて改良されてもよく、実際そうなっています。ここで注意しなければならないことがあります。チームがコンシューマからの個別の要求を取り入れたくなることもあるでしょう。しかし、コンシューマがビジネスロジックをデータプロダクトの提供者であるドメインに押し付けると、変更を加える際にプロバイダーとの間に強い依存関係が生まれてしまいます。これは、ドメインチーム間での非常に緊密な調整やバックログに関する交渉を引き起こします。ここでお勧めしたいのは、データプロダクトが特定のコンシューマのために作成されていないことを監督するガバナンス組織を設けることです。この組織は、ドメインチームを指導し、意見交換や知識共有の場を設け、実践的なフィードバックを提供し、チーム間の問題を解決する役割を果たします。

ビジネスロジックのアウトソーシングの危険性

筆者の経験談を紹介させてください。以前勤めていた職場で、経理部門の責任者が国際財務報告基準（IFRS）の会計ルールをすべてのデータプロダクトに組み込むことを提案しました。その人は、計算された結果を合計する方が、すべてを自分で計算するよりも簡単だと考えたのです。ビジネスロジックをほかのドメインにアウトソースしてはならない理由はたくさんあります。一つは、ほかのドメインが自分のドメインについて深い知識を持っていることを前提にしていることです。また、ビジネスロジックを更新しなければならない場合、すべてのドメインが同時に変更を実行する必要があります。何十、何百ものドメインがある環境で、これをどのように管理するのでしょうか。おそらく、これらすべての活動をオーケストレーションし、管理するために、大勢のコンサルタントが必要になるでしょう。ビジネスロジックは、あるべき場所に近づけるのが正しいアプローチです。IFRSの場合で言えば、正しく計算するために必要な情報を経理部門に提供するよう、ほかのドメインに依頼するのが正しいやり方です。その情報は、提供ドメインの元のコンテキストを保持しています。そして、ほかのドメインもそのデータを同じように利用できるからです。

4.3.8　欠測値、デフォルト値、データ型

欠損データ、低品質データ、デフォルトデータをどのように解釈すべきかについて、激しい議論を交わした経験があります。例えば、ある業務システムのWebフォームにユーザーの生年月日を入力する必要があり、ユーザーがそのデータを入力しなかったとします。もしも、入力ガイダンスが提供されていなければ、従業員は適当な値を記入するかもしれません。このような振る舞いによって、データが正確なのか、デフォルト値が入っているのかをコンシューマが

判断しにくくなるおそれがあります。例えば、誕生日のデフォルト値を9999-12-31などの不合理な値（未来の値）にするなど、分かりやすいガイダンスを提供することで、データの利用者は、データが正しく入力されていないことを確認することができます。

また、データセット全体を通して書式を統一しなければならないデータ型やデータに関して、ガイダンスを示すこともできます。例えば、小数点以下の桁数を適切に指定することで、データコンシューマがデータを使用する際に複雑なアプリケーションロジックを使う必要がなくなります。このような原則をシステムやドメインレベルで設定することもできます。一方で、大企業では、すべてのデータプロダクトに対して、どのようなデータ形式や型を使用しなければならないか、包括的なガイダンスを提供することもあります。

4.3.9　意味的一貫性

データプロダクトは、バッチ、イベントドリブン、APIベースなど、すべての提供方法において意味的に一貫していなければなりません。筆者にとっては当たり前のことなのですが、現在でもバッチ指向、イベント指向、API指向のデータに対してそれぞれ異なる内容の説明を提供している組織を見かけます。データの出所はどの流通パターンでも同じなのですから、データについての説明は、すべてのパターンで同じものにすべきだと思います。この点については7章でもう一度触れます。

4.3.10　不可分性

データプロダクトの属性は不可分でなければなりません。データプロダクトの属性は、最小レベルの粒度を表し、正確な意味を持たなければなりません。つまり、これ以上意味のある構成要素に分解できないものにしなければなりません。理想的な状態では、データ属性はデータカタログ内のビジネス項目と一対一にリンクされます。この場合のメリットは、データコンシューマがデータの分割や連結を行う必要がないことです[†4]。また、不可分性のあるデータを使用するメリットとして、ポリシールールをデータから切り離すことができるという点も挙げられます。例えば、法規制によって機微なデータに対するラベルについて考え直さなければならなくなった場合でも、すべての物理データのラベルを貼り変える必要がなくなります。適切なメタデータモデルと不可分性のあるデータがあれば、どのような変更があっても、ビジネスメタデータから自動的に反映されます。

[†4] 信じられないかもしれませんが、筆者はレガシーアプリケーションで、顧客の複数に関する値が、Firstname||Middlename||Lastnameのように二重パイプ演算子を使って1つのフィールドに連結されているのを見たことがあります。

4.3.11　互換性

データプロダクトは安定していなければならず、運用／トランザクションアプリケーションから分離されていなければなりません。そのためにはスキーマの変更を検知し、破壊的な変更を防止する仕組みが必要となります。また、バージョン管理や、場合によっては独立したパイプラインを並行して実行し、データコンシューマが、あるバージョンから別のバージョンに移行する時間を確保することも必要です。

データプロダクトの互換性を維持するプロセスは、それほど単純ではありません。古いデータプロダクトと新しいデータプロダクトの間で過去データを移動させる必要があるかもしれません。この作業は、マッピング、クレンジング、デフォルトロジックなどのETLタスクが含まれるため、複雑な作業になる場合があります。

4.3.12　抽象的な揮発性参照データ

複雑な参照値が、もっと抽象的なデータプロダクト向けの参照値に、どのようにマッピングされるかについて、ガイダンスを示すとよいでしょう。これには、変更速度が一致していない場合への配慮が必要となります。変更のペースが消費側のほうが速い場合は、消費側で複雑なマッピングテーブルを管理することを基本原則としなければなりません。提供側の変更ペースのほうが速い場合は、データプロダクトオーナーに対して、詳細なローカル参照値を、コンシューマに無関係で一般的な（粒度の粗い）参照値に抽象化または統合するよう求めるという原則が指針になります。この指針は、一般的な参照値をコンシューマに特化した参照値にマッピングするなどの追加作業を、コンシューマ側で行う場合があることも意味しています。

4.3.13　新しいデータは新しいオーナーシップを意味する

業務変革（ビジネスロジックを使用する意味論的な変更）のために作成され、流通されるデータはすべて新しいデータとみなされ、作成したドメインのオー

ナーシップの下に置かれます。新しく作成されたデータが共有される場合は、この章で説明するデータ流通の原則を適用する必要があります。

ユースケースに基づいた統合データを（コンシューマ整合の）データプロダクトとして扱うのは根本的に間違っています。ユースケース固有のデータをほかのドメインで直接消費できるようにすると、そのドメインはユースケースの実装に大きく依存することになります。そのため、代わりに抽象化レイヤーを必ず追加し、ユースケースをほかのコンシューマから分離してください。コンシューマ固有のデータを新しいデータプロダクトにするというこのアプローチにより、ドメインはほかのドメインから独立して進化することができます。

新たに作成されたデータを配布する際に懸念されるのは、トレーサビリティ（データに何が起こったかを知ること）です。透明性に関するリスクを軽減するため、データコンシューマには、取得したデータ、データに適用した一連の操作や変換をカタログ化するよう求めてください。このリネージのメタデータは中央で公開する必要があります。この点については10章と11章でまた触れます。

4.3.14　データセキュリティパターン

データセキュリティについては、データプロバイダーに対するガイダンスを規定する必要があります。このガイダンスには、以下の内容を含めてください。

- フィルタリングと行レベルのアクセスのために、メタデータをカプセル化することについてのガイダンス。データと共にメタデータを提供することで、ポリシーやビューを作成して、コンシューマがそれらの行を表示する権限を持っているかどうかに応じて、データの特定の行を非表示にしたり表示したりできます。
- 列レベルのアクセスおよび動的なデータマスキング用に、タグや分類に関するガイダンス。これらのタグや分類は、ポリシーやビューで使用できます。
- データを別々のテーブルに効率的に格納するためのガイダンス。粗視化されたセキュリティを提供し、性能を向上させ、保守を容易にします。

この話題については、8章で詳しく説明します。

4.3.15　メタモデルの確立

データプロダクトとそれに関連するメタデータを適切に管理するためには、一貫性のあるメタモデルを作成し、エンティティ（データドメイン、データプロダクト、データオーナー、ビジネス用語、物理データ、そのほかのメタデータなど）が互いにどのように関連するかを定義してください[†5]。メタモデルを作成する際には、データカタログを使用して、必要なメタデータをすべて収集するようにしましょう。例えば、カタログで新しいデータプロダクトをインスタンス化するたびに、データプロバイダーに説明を追加するように依頼し、データプロダクトをデータオーナー、出所のアプリケーション、データドメインなどに関連付けるワークフローを起動するようにしてください。次に、カタログはデータプロダクトの全要素をビジネス用語および技術的（物理的）データ属性に関連付けるようデータプロバイダーに依頼します。そして、これらの物理的な場所がアクセス可能かどうかをチェックする自動テストを実行しても良いでしょう。また、分類と利用制限についての情報を提供するよう依頼してもかまいません。

[†5] メタモデルとは、データを記述する際にどのようなメタデータを取り込む必要があるかを把握するための枠組みです。

4.3.16　セルフサービスの許可

データプロダクトとは、データの民主化を実現するものです。データプロダクトの作成体験を高めるために、データマーケットプレイスを構築し、ディスカバラビリティ、管理、共有、オブザーバビリティのために、セルフサービス機能を提供することを検討してください。例えば、エンジニアがデータカタログのRESTインタフェースを使用できるようにし、データプロダクトの登録を自動化できるようにします。適切に行えば、エンジニアは継続的インテグレーション/継続的デリバリー（CI/CD）パイプラインの一部としてデータプロダクトを登録できるようになります。この件に関する詳細なガイダンスは9章で提供します。

4.3.17　クロスドメインの関係

運用の世界では、システムが相互に絡み合ったり、強くつながったりすることがよくあります。顧客が注文を行う受注システムを想像してください。このようなシステムは、顧客データベースからのデータを保持しているはずです。そうでなければ、注文と顧客を結びつけることができないからです。このような依存関係を管理するには、データセット間の関係を記述し、データプロダクト間の結合キーや外部キーを参照するカタログやナレッジグラフを使用します。

ドメインがそれぞれほかのドメインから独立してデータプロダクトを開発する場合、コンシューマはどのようにして、あるデータが一緒になっている、あるいは関連していることを知るのでしょうか。このような依存関係を管理するには、データセット間の関係と、データプロダクト間の結合キーや外部キーを参照するカタログやナレッジグラフの使用を考えてください。

クロスドメインの（異なるドメインにわたる）関係を管理する際のガイダンスを示す場合、データがドメイン指向を貫くようにすることが重要です。データとデータオーナーシップの間の整合性はしっかりと確保されなければならないので、データがほかのドメインに配信される前に、クロスドメインの統合を行うべきではありません。

4.3.18　企業の一貫性

非中央集権化が進むと、データはそれぞれのドメイン内でサイロ化されます。すべてのドメインがそれぞれ個別にデータを提供し始めると、消費側でデータを統合して結合することが難しくなるため、データのアクセス性とユーザービリティに問題が生じます。このような問題を解決するために、企業の参照値（通貨コード、国コード、製品コード、顧客区分コードなど）に関するガイダンスを提供してください。このガイダンスで、企業の一貫性を確保することができます。場合によっては、データプロダクトオーナーに、ローカルの参照値を企業リストにある値にマッピングするよう求めてもよいでしょう。

データ専門家の中には、企業の参照値を使用することは、データメッシュの真の実装に準拠していないという意見もあります。これはある意味正しいのですが、参照整合性がなければ、すべてのドメインにわたって調和と統合のための作業が繰り返されることになります。多くのエンタープライズデータウェアハウスアーキテクチャで見られるような、企業全体の標準的なデータモデルを構築することを提案しているわけではなく、単にガイダンスとして参照データを提供するだけだ、ということに留意してください。結果として一貫性が得ら

れ、多くのドメインが同じ参照値を利用するようになり、大規模な組織では有効に働くでしょう。

同じガイダンスがマスター識別番号にも適用できます。このマスター識別番号は、マスターデータとローカルシステムのデータを結び付けます。これらのデータ要素は、どのデータがマスター登録され、どのデータが一緒になっているかを追跡するために重要です。そのため、ローカルドメインに対して、これらのマスター識別子をデータプロダクトに含めるよう求めてもよいでしょう。この分野のベストプラクティスは10章で紹介します。

4.3.19　履歴化、再配信、上書き

コンシューマのニーズに応えるため、データプロバイダーは多くの場合、過去のデータを元のコンテキストで保持する必要があります。例えば、コンシューマは次に何が起こるのかを予測するために、過去にさかのぼってトレンド分析を行い、このようなデータを必要とするかもしれません。これをうまく行うには、システムの設計、データの種類、後の分析のためにどのように履歴化すべきかを考慮して、ガイダンスを策定しておくべきでしょう。例えば、マスターデータを含むデータセットは、必ず、緩やかに変化する次元に変換されるべきです。過去データの処理については、「4.4.4　データの履歴化」で詳しく説明します。

4.3.20　複数のオーナーを持つビジネスケイパビリティ

ビジネスケイパビリティを共有する場合、共有データの取り扱い方法を定義しておくべきでしょう。これには、データプロダクト内で予約カラム名を使用してオーナーシップを定義したり、物理的に複数のデータプロダクトを作成したり、データプロダクト内にメタデータを埋め込んだりすることが考えられます。

4.3.21　運用モデル

データプロダクトを開発するには、効率的なデータ管理、標準とベストプラクティスの確立、性能モニタリング、品質保証を実現する運用モデルが必要です。一般的なデータプロダクトチームは通常、サポートアーキテクト、データエンジニア、データモデラー、データサイエンティストで構成されます。このチームがデータプロダクトを構築します。そして、データプロダクトを継続的に改善するためには、適切なサポートと資金提供が必要です。さらに、組織全

体でデータプロダクト構築のための標準とベストプラクティスを制定し、監督する組織体が存在しなければなりません。筆者は、組織が一番成功したと思えるときは、プレイブックを開発したときだと感じています。プレイブックとは、会社の目標、プロセス、ワークフロー、トレードオフ、チェックリスト、役割、責任のすべてを記述した文書のことです。このようなプレイブックは、オープンなリポジトリで公開され、誰もがそれに関与できるようになっていることが多いです。

適切な標準とベストプラクティスを確立するには、チームが性能と品質をどのように評価するかを定義することと、消費パターンごとに必要となるサービスをどのように組み合わせ、すべてのデータプロダクトで再利用できるようにするかを設計することが必要です。データプロダクトがコンシューマのニーズを満たし、継続して改善されていることを確認するために、チームは活動の価値と成功を評価しなければなりません。例えば、あるデータプロダクトの利用者数、品質に関する懸案事項や未解決事項、満足度、投資対効果、有効なユースケースなどで評価します。

さて、データをプロダクトとして管理する、というマインドセットの転換に必要となる基礎知識を学んだところで、次はアーキテクトの抱える疑問について取り組んでみましょう。データプロダクトを作るには、どのような戦略が良いのでしょうか。それは中央集権的に提供されるプラットフォームで実現されなければならないのでしょうか。それともドメインが独自のニーズに応えるようにしなければならないのでしょうか。

4.4　データプロダクトアーキテクチャ

アーキテクチャの複雑さを軽減するには、中央集権化と非中央集権化のバランスを適切に見極める必要があります。極端な例は、完全に連合化された自律的なアプローチです。それぞれのドメインが独自のアーキテクチャをホストし、独自のテクノロジースタックを選択できるようにします。しかし、これはデータプロダクト、インタフェースプロトコル、メタデータ情報などが乱立することにつながります。また、それぞれのドメインがアーカイブ、リネージ、データ品質、データセキュリティなどの重要な規律を正確に履行する必要があるため、データガバナンスとコントロールが非常に難しくなります。このようなことを、すべてそれぞれのドメインで実現することは非常に難しく、それは断片的でサイロ化された構造を生むことになります。

逆方向に極端な例は、中央集権的なアプローチです。単一のサイロという意味ではありません。複数のプラットフォームインスタンスやランディングゾーンブループリントで、同じ標準インフラストラクチャーサービスを共有する方法です。標準化されたコンポーネントと共有インフラストラクチャーは、コストを削減し、インタフェースの数を減らし、コントロールを向上させます。理想的には、複数のデータプロダクトアーキテクチャが同じプラットフォームインフラストラクチャー上でホストされますが、それらは互いに論理的に隔離されます。

4.4.1　高レベルのプラットフォーム設計

データプロダクトを構築するために必要なアーキテクチャとコンポーネントを紐解いてみましょう。データをプロダクトとして公開することで、データコンシューマはさまざまなドメインのデータを簡単に発見し、検索し、理解し、安全にアクセスできるようになります。とはいえ、データプロダクトの流通に必要となる共通プラットフォーム機能としては、データを取得し、データを読み出しに最適化されたバージョンに変換し、データをほかのドメインに安全に提供することが主要な機能です。データ駆動型意思決定というユースケースのために必要となる、分析機能やレポート機能は、後からアーキテクチャに追加します。これらについては11章で説明します。

もう少し詳しく説明すると、データプロダクトのアーキテクチャは、少なくとも以下の機能を備えている必要があります。

- 入力用のポートからデータを取得、インジェスト、導入する機能
- 出力用のポートから、さまざまなドメインにデータを提供する機能
- データのドキュメント化、発見、監視、保護、統制を行うためのメタデータ機能
- データプロダクトのコードを開発、配備、実行するためのデータエンジニアリング機能
- データ品質とマスターデータ管理機能
- 統一的なデータ購入体験を提供するデータマーケットプレイス機能

図4-5はデータプロダクトアーキテクチャの論理設計です。データがどのように取り込まれ、変換され、ほかのドメインに提供されるかを示しています。それぞれのコンポーネントの詳細については後ほど説明しますが、まずは概要を説明します。一番上にはメタデータレイヤーがあり、利用可能なデータ、そのプロパティ、ほかのデータ管理領域との関係についてインサイトを提供します。

このレイヤーの中には、オーケストレーション、オブザーバビリティ、契約管理などのメタデータリポジトリがあります。これらはデータマーケットプレイスとデータプロダクトアーキテクチャをつなぐ役割を果たします。

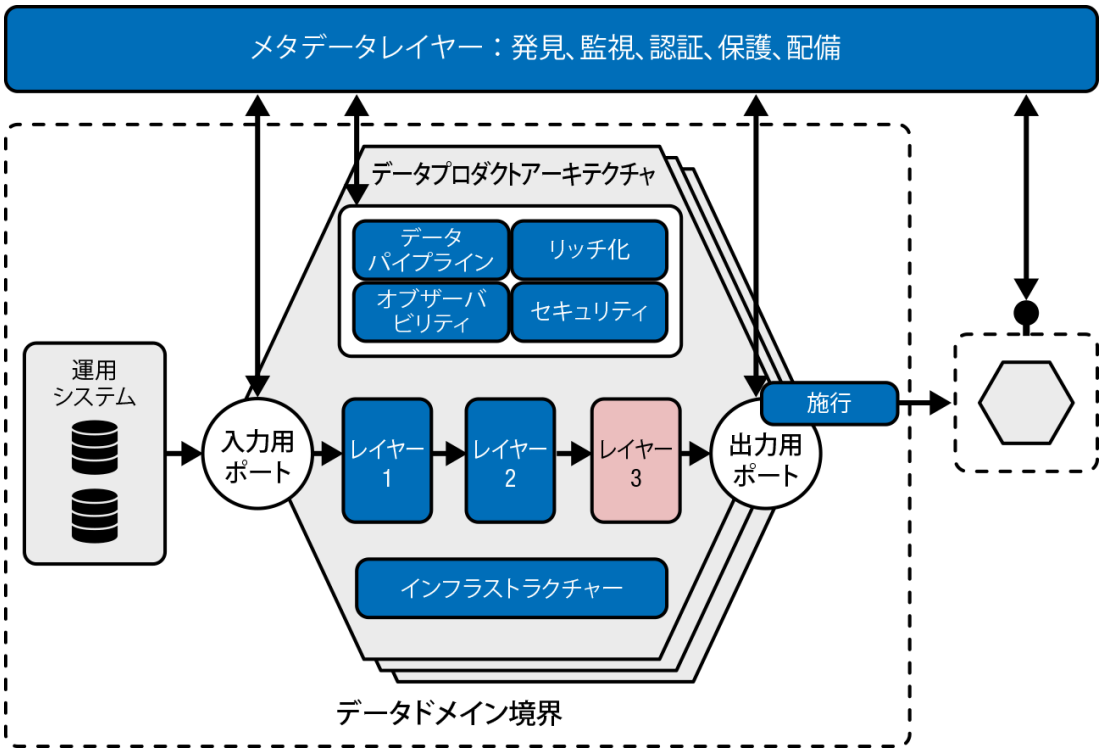

図4-5　データプロダクトアーキテクチャの例

インフラストラクチャーを共有する場合、異なるデータプロダクトアーキテクチャは隔離され、ドメイン間で直接共有されることはありません。それぞれのデータプロダクトアーキテクチャとその説明責任は、データプロバイダーの仕事のままです。データプロバイダー、つまりドメインは、専用のデータプロダクトアーキテクチャのインスタンスとパイプラインを持ち、データの品質と完全性について責任を負います。

共有プラットフォームとデータプロダクトアーキテクチャを設計する上で重要なことは、中心となる基本的かつドメインに依存しないコンポーネントのセットを提供し、さまざまな設計上の決定を行い、ドメインから繰り返し発生する問題を抽象化し、データ管理の課題を少なくすることです。次の項では、現場での経験に基づいた重要な検討事項を紹介します。

4.4.2　データの取り込みと導入のための機能

データの導入は、データアーキテクチャの構築の第一歩で不可欠です。ソースシステムからデータを取り出すプロセスも、大規模なデータ利用において、依

然として非常に難しい課題です。これにはいくつかの検討すべき点があります。

4.4.2.1　インジェスト方式

まず、データを配信したり同期したりするスピードを決めなければなりません。大きく分けると、2つの選択肢があります。

バッチ処理
定期的にしかデータを必要としないコンシューマは、バッチ処理（大量のデータを一度に転送するプロセス）を利用することができます。バッチは一般的に、ファイルとして保存された数百万レコードのデータセットです。なぜ今でもバッチ処理が必要なのでしょうか。多くの場合、これしかデータを収集する方法がないからです。主要な関係データベース管理システム（RDBMS）には、バッチ処理でデータの移動をサポートするバッチコンポーネントがあります。**データ照合**（データ移動中にデータを検証するプロセス）は、よく問題になります[†6]。特に財務プロセスでは、すべてのデータが揃っていることを確認することが不可欠です。さらに、多くのシステムは複雑で、膨大な数のテーブルがあり、データを選択し抽出するためにはSQL（Structured Query Language）しか方法がありません。

リアルタイムのデータインジェスト
データの逐次更新やほぼリアルタイムの更新を必要とするコンシューマは、**イベント駆動**または**APIベース**のインジェストを使うのが良いでしょう。このような処理により、速やかにデータを収集・処理することができます。データの量が比較的少なく、データに対する計算が比較的単純で、ほぼリアルタイムなデータが必要となるなユースケースに適しています。多くのユースケースはイベント駆動のインジェストで処理できますが、完全性や大量データが重要な課題である場合、ほとんどのチームは従来のバッチ処理を使うことになるでしょう。この点については、6章で詳しく説明します。

[†6] データ照合はさまざまな方法で実装できます。例えば、行数を比較したり、列チェックサムを使用したり（https://oreil.ly/NqCeO）、大きなテーブルをスライスして再処理したり、Redgateのようなデータ比較ツールを使用したりすることができます（https://www.red-gate.com）。

リアルタイムでのデータインジェストは普及しはじめていますが、ストリーミングのみ（Kappa）のアーキテクチャでは、すべてのユースケースのバッチ処

理を完全に置き換えることはないでしょう。ほとんどの企業では、両方のインジェスト方式が並行して使われつづけると思われます。

4.4.2.2　複雑なソフトウェアパッケージ

データ導入において、専用ソフトウェアパッケージがどれほど多岐にわたっているかを理解しておかなければなりません。ソフトウェアパッケージの多くは、理解するのもアクセスするのも非常に大変です。データベースのスキーマは非常に複雑であることが多く、参照整合性は通常、データベースではなくアプリケーションによってプログラム的に保守されます。ベンダーによっては、データを保護していて、サードパーティ製のソリューションを使わなければ、データを取り出せないものもあります。

このような複雑な状況では、ソースシステムごとに、データ抽出用のサービスを追加して、アーキテクチャを強化する必要があるかどうかを評価すべきです。このようなサービスを使えば、複雑なソースシステムの構造を隠蔽し、長期間にわたるコストのかかるマッピング作業をしなくてすむようになります。また、ベンダー製品の複雑なデータスキーマは、通常、直接的には制御できないため、密結合からも解放されます。ベンダーが製品のバージョンアップ版をリリースし、データ構造が変更されてしまえば、すべてのデータパイプラインが壊れてしまいます。要するに、サードパーティ製コンポーネントは投資に値するものなのです。

4.4.2.3　外部APIとSaaSプロバイダー

外部のAPIプロバイダーやSaaSプロバイダーも一般的に注意が必要です。完全なデータセットが必要であるにもかかわらず、SaaSプロバイダーがAPI経由で比較的少量のデータしか提供していない状況に遭遇したことがあります。また、プロバイダーによっては、スロットリング（リクエスト数に対するクォータや制限）をかけるかもしれません。さらに、呼び出しごとに高額な料金が課金される場合もあります。このような場合は、APIからのデータ抽出をサポートするツールを入手するか、定期的にデータを取得するカスタムコンポーネントを構築することをお勧めします。

カスタム実装を正しく設計すれば、すべてのAPI呼び出しを、まずデータプロダクトアーキテクチャにルーティングするでしょう。もし同じリクエストが直近に行われていれば、結果はデータプロダクトアーキテクチャから自動的に返されます。そうでない場合は、SaaSプロバイダーのAPIが呼び出され、結果が

返され、以降の呼び出しのために、データプロダクトアーキテクチャにもその結果が保存されます。このパターンを使用すれば、最終的には、データの完全なコレクションを作成してコンシューマに提供することができます。

4.4.2.4　リネージとメタデータ

データ抽出と導入を行うすべてのコンポーネントにおいて、データリネージの抽出に十分注意しなければなりません。組織においては、どのようなデータが抽出され、データにどのような変換が適用されたかを把握することが重要です。データリネージリポジトリを標準化し、コネクタをどのように統合するかを決めるのは、重要な作業です。何が統合され、何が統合されないかを理解するために、この作業を前もって行う必要があります。リネージは、変換フレームワークやETLツールに密接に関係しているため、後から追加するのは難しいからです。実際の実装を始める前に、選択し、テストし、検証し、標準化するのがベストです。

4.4.3　データ品質

データ品質も、設計において重要となる検討事項です。データセットがデータプロダクトアーキテクチャにインジェストされるとき、その品質は検証されなければなりません。これには2つの側面があります。

まず、公開されているスキーマと比較し、データの整合性を検証します。これは、新しいデータが高品質で、自社の標準や期待値に準拠していることを確認するためであり、そうでない場合は対策を講じる必要があります。この最初の防衛ラインは、データオーナー、データスチュワード、データエンジニアの責任です。つまり、データを作成し、提供するチームメンバーです。データが法規制や、データコンシューマが規定するデータ品質要件を満たすかどうかは、このメンバーにかかっています。

次に、本来の意味で連合的な品質チェックが存在します。この検証では、それぞれのデータプロダクトチームにあまり頼りません。なぜなら、何が良いデータ品質かを定義するのはほかのチームだからです。この種のデータ品質チェックは通常非同期に実行され、フラグを立てたり、各関係者に通知したりします。連合型のコントロールは、通常は機能的な確認です。完全性、正確性、一貫性、妥当性などです。データ完全性と連合型データ品質は別々の分野であり、独自のメタデータリポジトリを持ちます。また、そこにスキーマメタデー

タや機能検証ルールが格納されることもあります。このような場合、それぞれ独自の標準プラットフォームサービスを持ちます。

プラットフォーム設計において、データプロダクトアーキテクチャに共有インフラストラクチャーを使用するメリットは、データ品質処理にビッグデータの力を活用できることです。例えば、Apache Spark（https://spark.apache.org）は、数億行のデータを処理する分散処理能力を提供します。このような機能を効率的に使うために、データをドキュメント化し、テストし、プロファイリングするデータ品質フレームワークを使うことができます。例えば、Great Expectations（https://greatexpectations.io）やSoda（https://www.soda.io）は、データ品質のためのオープンスタンダードであり、多くの大規模組織で利用されています。共有インフラストラクチャー上でデータ品質を管理するもう一つのメリットは、クロスドメインでの参照整合性チェックを実行できることです。ソースシステムは、ほかのアプリケーションやシステムのデータを参照することがよくあります。参照整合性をクロスチェックし、異なるデータセットを比較対照することで、これまで気づかなかったエラーや相関関係を見つけることができます。

データ品質監視を適切に実装すれば、データ品質の問題を検出・監視するだけでなく、データ品質コントロールフレームワークの一部となります。このフレームワークでは、既存のデータに対して、新しいデータのチェックや追加を行います[†7]。何らかの理由で、指定されたしきい値より品質が低下した場合、フレームワークはデータを保留し、データオーナーに確認してもらい、そのデータを承認するか、削除するか、再提供するよう求めることができます。データ品質コントロールフレームワークを持つということは、品質の問題を継続して修正し、再発を防止するクローズドフィードバックループが構築されている、ということです。データ品質は常に監視され、品質レベルが低下した場合は直ちに対処されます。データ品質の問題は、データプロダクトアーキテクチャ内ではなく、ソースシステム内で解決されなければならないことを忘れないでください。データ品質の問題をソースで解決すれば、ほかの場所で問題が発生することはあり得ないからです。

[†7] Azure Synapse AnalyticsとMicrosoft Purviewを統合し、Great Expectationsを使用してデータをインテリジェントに処理・検証する方法については、筆者の記事“Modern Data Pipelines with Azure Synapse Analytics and Azure Purview”（https://oreil.ly/QFMXn）を参照ください。

データ品質が与える影響を軽視すべきではありません。もしデータの質が悪ければ、コンシューマはデータの整理と修正という作業を何度も繰り返すことになります。また、標準化の重要性も指摘したいと思います。もし個々のチームが独自のデータ品質フレームワークを採用すれば、ドメイン間で指標や結果を比較することができなくなります。

4.4.4　データの履歴化

過去データを正しく管理することは、ビジネスにとって非常に重要なことです。過去データがなければ、長期的なトレンドを把握し、将来の予測を立てることができないからです。過去データの管理には、標準化も検討する必要があります。この項では、その背景情報を説明し、このような過去データを管理するためのさまざまな表現方法とアプローチを紹介します。

過去データを整理することは、データウェアハウスの鍵となる部分です。このコンテキストでは、よく**不揮発性（nonvolatile）**と**時変系（time-variant）**という言葉を耳にします。不揮発性とは、新しいデータが追加されても以前のデータが消去されないことを意味します。時変系とは、データが一定期間内で一貫性を持つことを意味します。例えば、データは毎日あるいは定期的に読み込まれ、その期間内は変化しないということです。

大量の過去データを保存・管理する上で、データプロダクトは重要な役割を担っています。非中央集権型アーキテクチャではどのように対処するのでしょうか。データウェアハウスとの違いは、データプロダクトはデータを元のコンテキストのまま保存する点です。データプロダクトはドメインと整合しているため、エンタープライズデータモデルへの変換は不要であり、元の（ドメインの）コンテキストが失われることはありません。これは大きなメリットです。ドメインは、機械学習などの運用上のユースケースのために自身のデータプロダクトを「ドッグフード化」することができ、同時にほかのドメインにもデータを提供することができます。その結果、ドメインは自分自身が所有するデータを管理する、という必要性を本質的に持つことになります。最終的には、自分のデータプロダクトを利用するという、このやり方は、その品質と顧客満足度の両方を向上させることでしょう。

データプロダクトアーキテクチャはテクノロジーに依存しない概念です。ただし、こうしたアーキテクチャの多くは、データレイクサービスのような低コストの分散ファイルシステムで設計されることが多いでしょう。マスターデータ、トランザクションデータ、参照データの更新や上書きには、それぞれ異な

るデータ処理方法が必要になります。したがって、これから紹介するアプローチの1つまたは複数を組み合わせて処理することをお勧めします。それぞれのアプローチには、受信データへの投資や管理、データ消費の容易さとの間にトレードオフが存在します。どのアプローチにも長所と短所があります。

4.4.4.1　ポイントインタイム（Point-in-time）

過去データを管理するアプローチとして最初に紹介するのは、元データを過去参照として保存する方法であり、通常これは不変となります。例えば、コピー処理を使って、データをリザーバーにインジェストします。この挿入だけを行う方法では、データを論理的にパーティショニング（分割）すると良いでしょう。例えば、日付と時間を使って論理的に分割します（**図4-6**参照）[†8]。

[†8] パーティショニングは、管理性、性能、可用性を向上させるために、ファイルやテーブルを異なるグループ（パーティション）に分割する一般的な手法です。通常、地理的属性（都市、国）、値ベース属性（一意識別子、区分コード）、時間ベース属性（配信日、時間）などのデータ属性で行います。

配信日ごとにパーティショニングされたデータ

ID	名前	Email	配信日
1	John Snow	john.snow@example.com	1-1-2019
2	Brian Stark	brian.stark@example.com	1-1-2019
1	John Snow	john.snow@example.com	2-1-2019
2	Brian Stark	brian.stark@example.com	2-1-2019
3	Elis Smith	elis.smith@example.com	2-1-2019
1	John Snow	john.snow@example.com	3-1-2019
2	Brian Stark		3-1-2019
3	Elis Smith	elis.smith@example.com	3-1-2019

データは緩やかに変化する次元（タイプ2）に処理され保存され、
ユーザーは特定の値を選択できる

ID	名前	Email	開始日	終了日
1	John Snow	john.snow@example.com	1-1-2019	
2	Brian Stark	brian.stark@example.com	2-1-2019	2-1-2019
3	Brian Stark		3-1-2019	
3	Elis Smith	elis.smith@example.com	2-1-2019	

図4-6　全次元のスナップショットと、緩やかに変化する次元（タイプ2）でデータをパーティショニングした場合のデータ構造の例

完全なスナップショットのみを管理する方法は実装が容易です。ETLを実行するたびに、出力全体を不変のスナップショットとして保存します。その後、新たなデータが格納される場所を指定してテーブルスキーマを置き換えます。このコンテナが、すべてのポイントインタイムスナップショットのコレクションになります。このアプローチではデータが重複してしまうと主張する専門家もいるでしょう。クラウドストレージは昨今では比較的安価であることから、この点について筆者は問題視していません。また、完全なスナップショットであれば、再配信、つまり時間の経過とともに行われたすべての変更を保持するような、統合データセットを再構築することも容易です。ソースシステムが誤ったデータを配信したことを発見した場合、データを再送信し、パーティションを上書きすることができます。

提供されるデータに、必要な過去データがすでに含まれている場合は、ポイントインタイム形式を使用するしか方法がないのかもしれません。例えば、すべてのレコードに対して開始日と終了日がすでに含まれているような形式でデータが提供される場合などです。

完全なデータセットをスナップショットする場合の欠点は、（遡及的な）データ分析が難しくなることです。ある特定の期間同士の比較が問題となります。開始日と終了日がわからない場合、コンシューマはすべての過去データを処理し、保存する必要があるからです。例えば、3年分の過去データを処理するには、1000以上のファイルを順番に処理する必要があるかもしれません。データ処理には、データの大きさにもよりますが、多くの時間と計算能力が必要となります。このような場合には、次に紹介するインターバルアプローチを検討することもお勧めします。

4.4.4.2　インターバル

過去データを管理・表現するためのもう1つの方法は、すべてのデータが比較・処理できるように過去データセットを構築する方法です。例えば、**緩やかに変化する次元**（SCD、slowly changing dimensions）を使ってデータセットを作成すれば、時間の経過に伴うすべての変化を表すことができます[†9]。このプロセスでは、現在のデータ配信を以前の配信と比較する必要があるため、ETLが必要になります。比較後、レコードはオープン・クローズされます。このプロセスでは通常、更新された値には終了日が、新しい値には開始日が設定されます。この比較では、無効なデータはすべて取り除くようにしてください[†10]。

[†9] 緩やかに変化する次元とは、データウェアハウスにおいて、時間の経過とともに現在と過去の両方のデータを格納・管理する次元のことです。データ管理およびデータウェアハウスにおける次元テーブルは、タイプ0〜6と呼ばれ

るさまざまな方法で管理できます。これらの方法論はKimball Group（https://oreil.ly/NUUW1）によって紹介されています。

[†10] 例えば、フレームワークの中で、属性レベルの `exclude_from_delta_processing` パラメータを使用して、比較対象からデータを除外することができます。

データ仮想化を使ってデータプロダクトを設計できるのか

データ仮想化の話を持ち出すのは、履歴化の議論まで待っていました。データ仮想化とデータプロダクトの作成は、以下のような理由からあまり良い組み合わせとは言えません。

- データ仮想化はデータ・ライフサイクル管理を補完するものではありません。関連性のないデータを基盤システムから移動させることはできません。すべての過去データをOLTPシステムに残す必要があり、データ仮想化は長期的には高コストなソリューションとなります。
- データ仮想化レイヤーは、密結合を招きます[†11]。ソースシステムが変更されると、データ仮想化レイヤーもすぐに変更する必要があります。ビューやレイヤーを追加することも効果はありますが、どのような変更であっても、ソースシステムのオーナーと、データ仮想化レイヤーを管理するエンジニアの間で調整が必要なことには変わりありません。
- データ仮想化はネットワークに依存します。多くのネットワークとホップ（経由するネットワークデバイス）を持つ分散環境では、遅延が増大することが想定されます。さらに、ネットワークと結合してしまうため、ネットワークがダウンすると仮想化レイヤーも壊れてしまいます。
- データ仮想化は、基盤となるテクノロジー（通常はRDBMS）により制約を受けます。データ仮想化は多くのデータベースシステムを読み出すことができますが、一般的にはドキュメント、キー/バリュー、カラムナ、グラフデータベースのエンドポイントを作成することはできません。
- データへのクエリ時に変換がリアルタイムで実行されるため、大量かつ何度も行われるクエリでは、データ仮想化はさらに多くの計算パワーを使用することになります。キャッシュ技術を使えばこのような問題は低減できますが、データを消費するために前処理を行う場合、必要な計算パワーは必ず大きくなります。

要するに、データプロダクトへのアクセスは仮想化できますが、データ仮想化エンジンのみを使用してデータプロダクトを設計・開発すべきではありません。

[†11] データベースベンダーの中には、データ仮想化レイヤーとも呼ばれるデータベース（仮想）クエリレイヤーを提供しているものもあります。このレイヤーはデータベースを抽象化し、データを最適化して読み出し性能を向上させます。また、セキュリティを向上させるためにクエリをインターセプトすることも、抽象化する理由として挙げられます。

インターバルを比較し、過去データを構築するアプローチには、データを処理し、重複を排除し、結合を行うため、データを効率的に格納できるという利点があります。図4-6でわかるように、下側の緩やかに変化する次元では、行数が半分になっています。その結果、例えば、関係データベースを使用する場合、データのクエリが簡単かつ高速になります。データのクレンジングや特定のレコードの削除（GDPR遵守のために必要な場合）も、これまで配信されたデータセットをすべて解析する必要がないため、簡単になります。

しかし、緩やかに変化する次元を構築するには、管理コストが増えるというデメリットがあります。すべてのデータを処理しなければならないからです。ベースとなるソースデータが変わった場合も、すぐに検知し、処理する必要があります。こうした検知を行うには、コードと計算能力が新たに必要となります。また、データコンシューマの履歴化に対する要件は、一般的にそれぞれ異なっていることも考慮に入れてください。例えば、データが日次ベースで処理されているにもかかわらず、コンシューマが月次での表現を必要とする場合、新たな処理工程が必要です。

さらに、誤ったデータが処理され、それが次元の中に入ってしまうことがあるため、再配信プロセスには手間がかかり管理も難しくなります。これは、再処理ロジック、バージョンの追加、有効期間などで対応できますが、それでも管理の負荷は増えます。ここでの問題は、データを正しく管理することが中央チームの責任になる恐れがあることです。このため、このようなシステムのスケーラビリティを実現したい場合は、インテリジェントなセルフサービス機能を数多く用意する必要があります。

また、すべてのコンシューマにとって汎用的に利用できるような過去データを構築することには、コンシューマが選択時に列を削除すると何らかの処理が必要になるというデメリットもあります。例えば、コンシューマが選択範囲を絞った場合、レコードが重複してしまい、重複排除のためにもう一度データを処理することが必要になるかもしれません。スケーラビリティとコンシューマのために、2つのアプローチを組み合わせて、完全な次元のスナップショットを保持した上で、「サービスとして過去データ」を提供することを検討してもよいでしょう。この方法では、小規模な計算フレームワークをすべてのコンシューマドメインに提供し、コンシューマドメインは、必要なスコープ（日次、週次、月次)、タイムスパン、属性、データセットに基づいて、過去データの作成を依頼することができます。パブリッククラウド上の一時的な処理インスタンスを使えば、費用対効果を高めることもできます。このアプローチの大きなメリットは、再配信の際に柔軟な対応ができる一方で、データセットの管理や準備にエンジニアリングチームを必要としないことです。もう1つのメリットは、タイムスパン、スコープ、属性を全員のニーズに合わせて調整できることです。

4.4.4.3　追加のみ

データ配信の形式によっては、追加のみのアプローチがベストな場合もあります。この方法では、アプリケーションデータベースから新しいレコードや変更されたレコードだけを読み込み、既存のレコードセットに追加します。これはトランザクションデータだけでなく、6章で説明するイベントベースのデータにも使えます。この場合、標準テーブル、ディレクトリテーブル、ビューで行われた変更を検知して取り込むことで、変更データのストリームを作成します。

履歴化とストリーミングインジェスト

リアルタイムのデータインジェストは、一般的に2種類の方法で処理されます。まず1つ目の方法では、データが入力されるとその場で変換が行われ、その結果が例えばNoSQLデータベースに直接追加されます。このプロセスは通常、数秒から数分で完了します。データが到着したらすぐにサービスを提供することが主な目的です。2つ目の方法は、リアルタイムのデータをマイクロバッチなどで定期的に処理する方法です。ここでの主な目的は、すべてのデータをマージして履歴化し、データをタイムリーかつ高速に下流で消費できるよう

にすることです。データをマージする際に注意しなければならないのは処理の遅延です。処理には時間がかかるからです。

4.4.4.4　履歴化戦略を定義する

データプロダクトに適した履歴化の方法は、データの種類、コンシューマの要件、法規制によって変わります。データプロダクトの履歴化と構築のための手法はすべて組み合わせることができ、データの種類によってさまざまな手法を使用することができます。例えば、マスターデータは、緩やかに変化する次元という形式を使用して、アプリケーションから配信・構築することができます。参照データはスナップショットを使用して簡単に処理できますが、同じシステムのトランザクションデータは追加のみの配信を使用して処理しても良いでしょう。すべての配信について完全なスナップショットを保存しながら、緩やかに変化する次元を構築することも可能です。

データプロダクトの開発と設計をうまく進めるには、総合的な戦略が重要です。ベストプラクティスは、それぞれのドメイン内でデータをレイヤー化し、履歴を構築する方法に集中することです。このレイヤー化には、変更データキャプチャサービスなどのスクリプトや標準サービスの開発と保守が必要になる場合もあるでしょう[†12]。

[†12] 筆者のブログ記事（https://oreil.ly/srK-Z）に、Sparkプールを使用してSCDテーブルをDeltaフォーマットで処理するための簡単なサンプルスクリプトがあります。

ここで取り上げたベストプラクティスは、見識を深めるためのものであると同時に、データプロダクト管理は大変な仕事であることを伝えるためのものでもあります。次の節では、アーキテクチャを開発する際のソリューション設計についてもう少し深く掘り下げます。すでに説明したことを踏まえつつ、実例を用いて説明します。

4.5　ソリューション設計

いくつかのベストプラクティスと設計原則を学んだところで、次は、より実践的なことを説明します。アプリケーションの複雑なデータを、ソリューションとサービスを使用して、読み出し最適化して抽象化する作業に取り掛かりまし

ょう。データプロダクトを開発するために組織が採用するテクノロジーや方法論は、通常、企業によって大きく違っています。また、データプロダクトのライフサイクル全体を管理するプラットフォームや製品が存在しないため、この問題を解決するのは容易ではありません。エンドツーエンドのデータプロダクトアーキテクチャを実装するさまざまな方法をすべて説明するには、もう1冊本が必要になってしまいます。そこで、具体的な例を取り上げる前に、主要な点に絞って説明することにします。

- クラウドは、オンプレミス環境に比べてメリットが多いため、大規模なデータ処理を行うためのデフォルトのインフラストラクチャーとなっています。一般的なクラウドプラットフォームはいずれも、簡単に実装できるように、一連のセルフサービス型のデータサービスを提供しています。
- データレイクサービスは、どのような組織でも人気のある選択肢です。データ量が日々増加する中、例えばHDFS互換のクラウドオブジェクトストレージにデータを格納する方法は、アーキテクチャを費用対効果の高いものにしてくれます。このような類のサービスの利点には、ストレージと計算を分けることや、軽量なクエリサービスを利用することによってデータの重複を減らせることなどがあります[†13]。
- 一般的によく使われるスタックは、Spark、Python、ノートブックによるデータ処理です。この方法を採用する理由としては、広く知られている活動的なコミュニティ、オープンソースと相互運用性のメリット、データエンジニアリングからデータサイエンスまでさまざまなユースケースへの幅広いサポートが挙げられます。SparkはETLや機械学習には適していますが、低遅延で対話的なクエリの実行にはあまり適していません。ノートブックを使用する場合には、データの履歴化、技術的なデータ変換、スキーマ検証など、繰り返し行われる作業に注意してください。ノートブックを使用する際のベストプラクティスは、送信先、検証、セキュリティなどの構成可能な手順を使用して、共通の手順とメタデータ駆動型のフレームワークを設計することです。[†14]
- 多くの組織では、データ変換とモデリングのために、dbt（https://www.getdbt.com）などのサービスをアーキテクチャに追加しています。Sparkをカスタムコードを使ってデータ変換に使うこともできますが、大規模なプロジェクトでは、テンプレート化したり構成設定を書いたりする方が効率的だと感じることが多いようです。テンプレート化ツールとノートブックはどちらも選択肢として有効であり、それぞれ補完し合うこともできます。データの検証、技術的な変換、履歴化をまずノートブック

で処理し、次にdbtのようなツールでデータを統合する企業をよく目にします。

- データのインジェストから提供までのプロセスを管理するには、オーケストレーションとワークフローの自動化が必要です。オブザーバビリティのためには標準化が重要ですが、それぞれのチームが独自のローカルの知見を活用し、ローカルでの優先事項に対処しやすいように、チームに自由度を与えることがベストプラクティスです。また、CI/CDとワークフロープロセスを統合しつつも、アプリケーションやデータプロダクトごとにパイプラインを別々にすることもベストプラクティスです。
- ツールを選ぶ際には、**最新のデータスタック**を検索することをおすすめします。オープンソースからクローズドソースのソリューションまで、さまざまな人気のある選択肢から選んでください。
- データカタログやデータリネージツールなどのメタデータ管理サービスは、データプロダクトアーキテクチャの外側に配置される場合が多いです。これらのサービスは中央組織が汎用サービスとして提供します。この点については9章で詳しく説明します。

[†13] ストレージと計算能力を分離することで、ワークロードの伸縮に合わせてインフラストラクチャーを自動的に拡張することができます。

[†14] 筆者のブログ記事“Designing a Metadata-Driven Processing Framework for Azure Synapse and Azure Purview”（https://oreil.ly/ZSpWX）を参照してください。

データプロダクト設計に関してよく聞かれるのは、データレイクとデータメッシュの概念はどのように関連しているのかということです。一見すると矛盾しているように見えるかもしれませんが、データレイクの基礎となるテクノロジーはデータプロダクト設計の概念を補うものです。つまり、データメッシュでデータレイクのテクノロジーを使うことは何も問題ありません。

ほかにもよく聞かれる質問として、ドメインごとに独自の最新データスタックを選択する自律性を持たせるかどうかというものがあります。マニアはすぐに、マイクロサービスとデータメッシュアーキテクチャを並べて、完全に自律的にすれば、それぞれのドメインが最適なテクノロジーを選択できると主張し

ます。しかし、多くの組織が失敗するのは、ドメインチームが相反する決断を下すからです。例えば、25個の「自律的な」チームがあり、それぞれが独自のツールや作業方法を用いてデータプロダクトアーキテクチャを実装しているとしましょう。もしすべてのツールが異なっていたら、リネージ、データ品質メトリクス、監視統計、マスターデータ管理の結果をどのように収集するのでしょうか。それぞれのチームが独自の標準を使用している場合、プラットフォーム間の相互運用性はどのように実現できるのでしょうか。これらの疑問はこの章の最後で説明しますが、ID、ガバナンス、相互運用性は、連合パターンの基礎となるものです。これらの要素のどれか1つでも欠けると、すぐにポイントツーポイントのソリューションが多数存在することになり、アーキテクチャのコストが高くなり、ガバナンスも難しくなります。自律性は、エンタープライズアーキテクチャとともに、中央管理レベルで始まり、標準化を必要とします。柔軟にするためには、疎結合にし、依存関係をなくすことです。自律性を重視した結果、組織の混乱、制御不能なテクノロジーの増殖、コストの爆発などをもたらしてはなりません。

4.5.1　実世界での例

この項では、実際に、データプロダクトアーキテクチャを作成する例を説明します。この例はそれほど複雑なものではありませんが、後述の議論や評価に欠かせない要素を含んでいます。ある組織のためにデータプロダクトアーキテクチャを設計すると仮定しましょう。この例では、約半分のドメインが運用とトランザクションという特性を持ちます。これらのシステムはゴールデンソースとして分類され、データインジェストの起点となります。そのほかのドメインはコンシューマです。そして、データはこのコンシューマに提供されなければなりません。

スクリーンショットや詳細な手順など、この例の詳しい説明については、筆者のブログ記事“Using DBT for Building a Medallion Lakehouse Architecture”（https://oreil.ly/PNhEt）をご覧ください。

もし筆者がAzure上でこの組織のデータプロダクトアーキテクチャの設計を任されたとしたら、図4-7のようなソリューションを提案するでしょう。まず、最初のドメインのために、データランディングゾーン（https://oreil.ly/J5Mbb）と、標準化されたサービスセット（Azure Data Factory（ADF）、Azure Data Lake Storage（ALDS）、Azure Databricks）を持つリソースグループをプロビジョニングします。次に ADFなどのサービス（https://oreil.ly/mSyb7）を使ってデータを収集し、蓄積します。可能であれば、既製のコネクタを使用します。または、アプリケーションオーナーに対して、中間的な場所にデータをエクスポートし格納するよう依頼します。例えば、Blob Storageアカウント（https://oreil.ly/MFBg2）などが考えられます。後の処理では、ここからデータを取り出します。

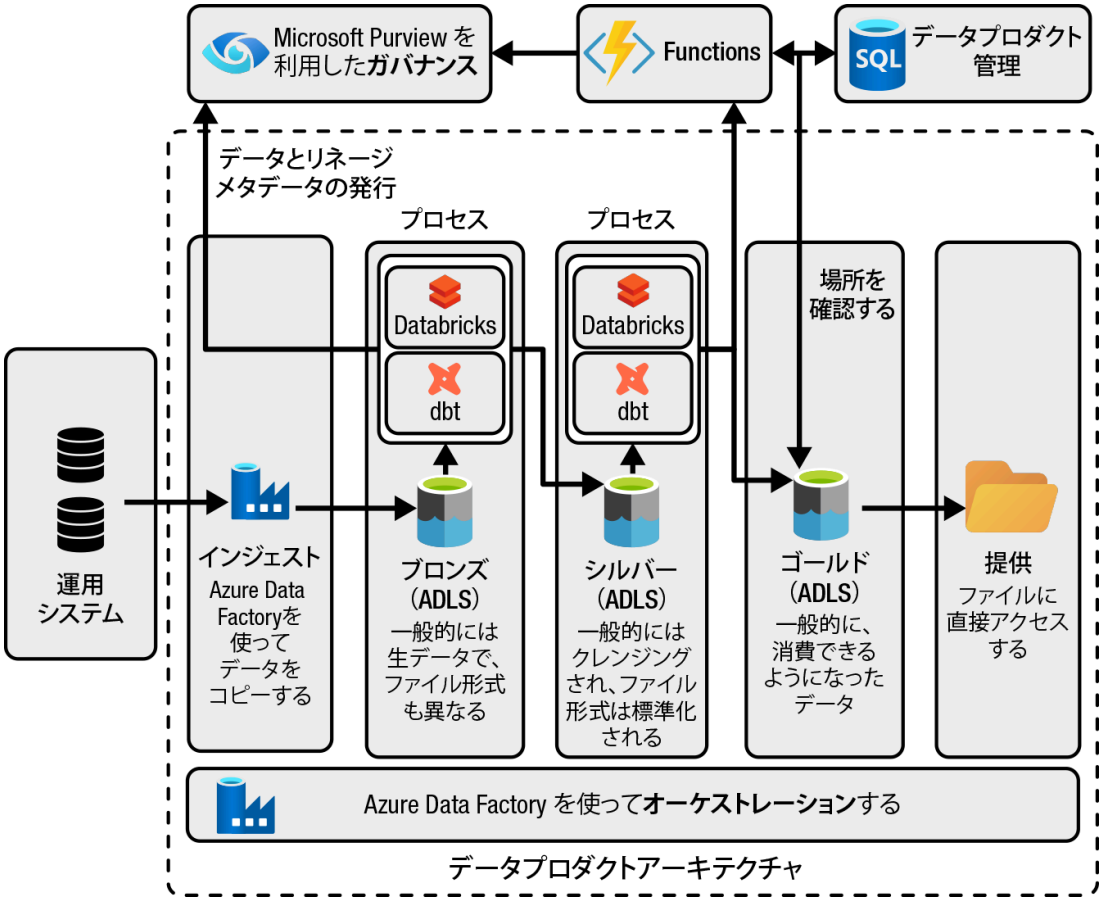

図4-7　レイクハウス設計を用いたシンプルなデータプロダクトアーキテクチャ。データのインジェスト、データプロダクトの作成、データガバナンスのためのサービスを備えている

筆者なら、ADLS（https://oreil.ly/RNly0）を使って、データプロダクトアーキテクチャにデータを格納するでしょう。データをグループ化する際の代表的なデザインパターンは、3つのコンテナを使用する方法です[†15]。

ブロンズ
複数のフォーマットや構造を持つ生データ。

シルバー
フィルタリングされ、クレンジングされたデータ。標準化された列指向のデータファイルフォーマットを使用して保存されます。

ゴールド
ほかのドメインですぐに消費できるように読み出し最適化されたデータ。組織によっては、このような読み出し最適化されたデータセットやコンシューマ向けのデータセットをデータプロダクトと呼びます。

[†15] データレイクの構築を計画しているのであれば、“Hitchhiker's Guide to the Data Lake”（https://oreil.ly/RVWEb）をご覧ください。ガイダンスとベストプラクティスをまとめたすばらしいリソースです。

データレイクを設定した後、最初のデータパイプラインを作成します。まず、コピー処理（https://oreil.ly/PIY2k）により、データをブロンズレイヤーにコピーします。このレイヤーのデータは、バージョン管理が不要なのでParquet形式で保存します。データの履歴化には、YYYYMMDDのパーティショニング方式を使います。アドホック分析やデバッグのために、古いバージョンのデータを保存しておきます。

次に、別のパイプラインを追加して、ノートブックを実行し、Databricksにパラメータ（https://oreil.ly/T3nd3）を渡します[†16]。Databricksは動的にノートブックを使用してデータを検証し、既存のスキーマを削除して、YYYYMMDD形式のパーティションに新しいスキーマを作成します。

[†16] MicrosoftはOpenLineageプロジェクトと共同でソリューションアクセラレータを開発しました。このオープンソースのコネクタは、Azure DatabricksのSpark処理からMicrosoft Purviewにリネージメタデータを転送し、テーブルレベルのリネージグラフを表示してくれます。このプロジェクトはGitHub（https://oreil.ly/uKwN4）で公開されています。

シルバーレイヤーでは、高速で腐敗からの保護を提供するDeltaファイル形式が筆者のお気に入りです。変更をロールバックして以前のバージョンにデータ

をリストアする必要がある場合は、データ保持機能（https://oreil.ly/RN182）が使えます。Databricksは、データを変更する操作ごとに新しいバージョンのテーブルを作成し、デフォルトで30日分の履歴を保存します。ブロンズレイヤーからデータを選択し、タイプ2のSCDテーブル設計を使用してデータを変換する場合、筆者はdbtを使用します。データを選択する前に、データが高品質で整合性の標準を満たしていることを確認するために、一連のテストを行います。このレイヤーでもう1つ注意することがあります。ソースが異なる場合、それらを別々のデータフィードとして扱うようにしてください。すべてのデータを統合して組み合わせるのは、次のレイヤーで行います。

データはゴールドレイヤーに移され、そこで統合され、コンシューマから使える形になります。すべてのオブジェクトを使いやすく、消費可能なデータセットにマッピングするために、ビジネスロジックを作成する必要があります。ここでもdbtテンプレートを使い、論理データエンティティごとにテンプレートとYMLモデル（https://oreil.ly/cO1JP）を作成します。それぞれのテンプレートでは、適用する選択、結合、変換を定義し、一意のフォルダ（顧客データ、注文データ、売上データなど）に書き込みます。それぞれの出力は、対象領域を中心として論理的にデータをまとめます。データはマテリアライズされ、ファイル形式はここでもDeltaです。

最後に、ADFにパイプラインを保存し、スケジューリングします。それぞれのステップについて、ステップを追加して、読み込み日時、プロセスID、処理したレコード数、エラーや成功のメッセージなどを受け取るようにします。

ドメインが1つ増えるごとに、これらのステップをすべて繰り返します。それぞれのドメインは、そのドメイン独自のデータプロダクトアーキテクチャにデータプロダクトを格納し、独自のデータパイプラインを管理し、データプロダクトの構築に同じようにレイヤー化を施します。このようなことをコントロールしながら大規模に行えるようになったら、この設計にバリエーションを増やしていきます。例えば、同じ意味のデータを、さまざまなコンシューマ向けにさまざまな表現で提供できるよう、出力ポートをいくつか用意してください。

アーキテクチャのガバナンス、セルフサービス、セキュリティの部分については、Azure SQLデータベース（https://oreil.ly/nuZK3）、Azure Functions（https://oreil.ly/oWukr）、統合データガバナンスサービスであるMicrosoft Purview（https://oreil.ly/fPmLs）の組み合わせを提案します。これらのコンポーネントは、データ管理ランディングゾーンに配備し、すべてのデータプロダクトのカタログ化と監視に用いられます。

データプロダクトを公開する際には、すべてのドメインに対して、CI/CDパイプラインの一部として、関数を用いてデータプロダクトを公開するよう依頼します。例えば、この関数はゴールドレイヤーからdbt YML仕様を入力として受け取り、これらの仕様をSQLデータベースに格納します。次に、この関数は、提供されたロケーション情報とスキーマ情報が、データプロダクトのURI（Universal Resource Identifier）と構造を表しているかどうかを検証します。表していれば、さらに関数を呼び出して、すべての情報をカタログに公開します。この最後の呼び出しによって、データがほかのコンシューマに公開されます。

コンシューマに対してデータにアクセスしてもらうために、筆者であれば、セルフサービスのデータ検出とアクセス（https://oreil.ly/zjeeW）の利用を提案するでしょう。裏では、個々のデータプロダクトに対するアクセス制御リスト（ACL）（https://oreil.ly/bsl22）を作成します。ACLは列レベルや行レベルの細かいフィルタリングには向いていませんが、この例では問題ありません。より複雑なケースについては8章で説明します。

この例は説明のために単純化されています。実際には、セキュリティ、プロファイリング、複雑な検索、データのインジェストと提供方法の多様性、さまざまなリッチ化シナリオなどの要素を考慮する必要があるため、データプロダクト管理はもっと複雑です。例えば、データのアーカイブ、トークン化、一定期間内のデータへのアクセスなど、特別な制約事項があるかもしれません。

この例から重要なポイントを簡単に振り返ってみましょう。第一に、データプロダクトを作成するための基盤をすばやく構築しました。ドメインはそのコントロール下にあるデータのライフサイクルと、アプリケーションによって取り込まれた生の状態から、外部のパーティが消費できるようにデータを変換する責任を負います。第二に、このアーキテクチャは、データプロダクトアーキテクチャを備えたドメインを追加することで、容易に拡張することができます。第三に、ドメイン指向のデータオーナーシップ、data as a productという考え

方、セルフサービスデータプラットフォーム、連合型計算ガバナンスなど、データメッシュの原則をいくつか適用しました。次の項では、大規模化する際に考慮すべき点について説明していきます。

4.5.2　ストレージアカウントとの整合性

データプロダクトデータとストレージアカウントの整合性に気を配ることが重要です。前の項の設計例では、データプロダクトアーキテクチャを、単一ドメインの複数のデータプロダクトデータセットで共有することにしました。ほかの方法として、それぞれのデータセットを、専用のストレージアカウントとデータ処理用のSparkプールと組み合わせることもできます。このような構成は、データ、コード、メタデータ、必要なインフラストラクチャーをまとめることになり、データメッシュにおけるデータプロダクトの概念に近づきます。アクセス管理がシンプルになり、自動化が可能になるため、処理クラスタとストレージアカウントをその場でプロビジョニングできます。例えば、シルバーレイヤーが一時的なものであれば、不要になった時点で消去することができます。一方、データプロダクトとインフラストラクチャーを密接にリンクさせると、インフラストラクチャーのオーバーヘッドと管理対象となるパイプラインの数が急激に増えます。例えば、管理、セキュリティ、スキャン、監視が必要となる、多数のアクセス管理やネットワーク、サービスのエンドポイントが必要になります。また、データプロダクトが同一データから作成されることが多いため、ストレージアカウントが複雑に絡み合う可能性があります。そのため、データプロダクトとインフラストラクチャーをどの程度密に連携させるかを検討する必要があります。

複数のドメインで同じストレージアカウントを共有することはできるのでしょうか。これは可能ですが、複数のドメインで基盤となるストレージを共有するには、異なるコンテナとフォルダ構造が必要になります。これは小規模な組織でよく見かけるパターンです。なぜなら、ストレージを一元管理することで、データの移動やセキュア化のために生じる管理負荷などの問題が軽減されるからです。一方で、すべてのドメインでACLやIPアドレスなどの制限や設定項目を共有しなければなりません。また、地理的な冗長性や性能を柔軟に調整する必要がある場合には、最適な方法とは言えないでしょう[†17]。繰り返しますが、トレードオフを考慮する必要があります。

[†17] 地理的な冗長性は、重要なコンポーネントやインフラストラクチャー（サーバーなど）を地理的に異なる複数のデータセンターに分散させることで実現されます。例えば、クラウドストレージでは、従来のハードディスクと高速なソリッドステートドライブを選択できます。

もしくは、ドメインローカルと中央の両方のストレージアカウントを使用するハイブリッドアプローチを導入することで、ローカルと中央のデータのバランスを取ることもできます。この場合、それぞれのドメインは専用ストレージアカウントを使用し、そこでは（例えば、ブロンズやシルバーレイヤーで）すべてのドメイン固有の処理が行われます。このようなアカウントは内部でのみ使用され、ほかのドメインに公開されることはありません。これらのドメインローカルなストレージアカウントと並んで、（ゴールドレイヤーで作成された）データプロダクトの最終データをほかのドメインに流通させるための中央共有ストレージアカウントがあります。この共有ストレージアカウントは多くの場合、中央部門が管理します。このように設計しているのは、すべてのデータプロダクトのデータ交換を監視しながら、ドメインに柔軟性を提供するためです。

4.5.3　データパイプラインとの整合性

もう1つ重要な点は、データパイプラインとデータプロダクトの整合性です。前述したように、データパイプラインはデータを移動したり変換したりするための一連のステップです。一般的には、メタデータ駆動型のインジェストフレームワークと同じように、メタデータとコードを使用して、どのデータに対してどの処理を実行するかを決定します。前述の例では、すべてのデータプロダクトに同じパイプラインを使用しましたが、専門家によっては、それぞれのデータプロダクトに専用のパイプラインを使用すべきと主張する方もいます。問題は、どちらの方法が読者の状況にふさわしいかということです。

この質問に答えるために、データプロダクトの粒度とこれに対応するデータパイプラインについて考えてみましょう。あるアプリケーションのデータをバッチ処理し、そこから複数の粗視化データプロダクト（データセット）を構築し、それぞれ独自のデータパイプラインを使用するとします。データプロダクトの入力データは重なることが多いため、複数のパイプラインが、同じソースシステムのデータを処理する可能性があります。つまり、処理のステップが繰り返される可能性があるため、保守が難しくなります。さらに、データの特定

の部分を異なるパイプラインで異なる間隔で移動させると整合性に問題が生じ、コンシューマが後でデータを結合するのが難しくなります。

データプロダクトとデータパイプラインの設定がきめ細かすぎると、それもまた難しいことになります。基盤となるソースシステムが、それぞれ独自のデータパイプラインを持つような何百もの小規模なデータプロダクトを生成する場合、問題のトラブルシューティングや依存関係の監視が難しくなり、複雑な処理ステップが繰り返されることになります。さらに、コンシューマがデータを利用する際にも、多くの小さなデータセットを統合する必要があるため作業が大変になります。多くのパイプラインが微妙に異なる間隔で実行されることが想定されるため、参照整合性も大きな問題となります。

結論として、データプロダクトごとに専用のパイプラインを持つことは、一般的にはあまり理想的なソリューションとは言えません。ベストプラクティスとしておすすめするのは、一連の処理を適切に行うパイプラインを1つ作成し、アプリケーションの状態全体を取得し、コピーし、データを使いやすいデータセットに変換することです。

4.5.4　データ提供のための機能

データプロダクトアーキテクチャのデータ提供側へ話を移し、そこでどのようなコンポーネントとサービスが必要になるのかを見てみましょう。出力ポートからデータを提供する場合[†18]、データ処理にはさまざまな側面があり、データプロダクトアーキテクチャの設計方法にも影響を及ぼします。データプロダクトアーキテクチャの設計は、消費ドメインのユースケースと要件に大きく左右されます。多くの場合、一刻を争うようなデータを処理しているわけではありません。質問への回答やビジネス上のニーズを満たすには、数時間から数日かけることもあります。しかし、数秒以内にデータを配信しなければならないユースケースもあります。

[†18] データメッシュでは、標準化されたインタフェースを使用してデータプロダクトを共有し、読み出し専用および読み出し最適化されたデータへのアクセスを提供することを推奨しています。これらのインタフェースは**出力ポート**とも呼ばれます。

データプロダクト管理を、ビジネスインテリジェンスや機械学習アプリケーションなどのデータ価値創造と混同してはいけません。重なる部分もありますが、一般的にコンシューマ側ではデータの利用方法が多岐にわたっています。したがって、データプロダクトの作成はデータ価値の創出とは切り離して管理する必要があります。

近年、ストレージやデータベースサービスの種類は飛躍的に増加しています。従来、トランザクションシステムや運用システムは、完全性を重視して設計されているため、ACIDに適合した関係データベースを使用していました。しかし、ACIDに適合しないスキーマレスデータベース（ドキュメントストアやキー/バリュー型ストアなど）もトレンドとなっています。これは、性能を向上させるために、整合性管理を緩和し、モバイルデバイス、ゲーム、ソーシャルメディアチャンネル、センサーなどから発信されるデータなど、さまざまな種類のデータに柔軟に対応できるようにするためです。データのサイズにも差があります。比較的少量のデータを保存するアプリケーションもあれば、テラバイト単位のデータを使用するアプリケーションもあります。

なぜこれほど多くの種類のデータベースが存在するのか

データベースを選択する際には、多くの要素とトレードオフを考える必要があります。データの構造だけでなく、一貫性、可用性、パーティション耐性、パフォーマンス向上のためのキャッシュやインデックス作成など、さまざまなニーズを見極める必要があります。データの保存と取り出しには、小さなチャンク、大きなチャンク、ソートされたチャンクなど、さまざまな方法があります。分散性と一貫性のトレードオフは、継続的な監視や分析などの機能と同じように、性能に影響を与える可能性があります。また、ベンダーロックイン、サポート、互換性、クエリ言語など、非機能要件も考慮する必要があります。つまり、どのデータベースもあらゆる面で優れているわけではないということです。

データプロダクトアーキテクチャを使用する利点は、複数の設計を追加して、さまざまなユースケースに特有の読み出しパターンに対応できることです[†19]。さまざまなデータ構造、速度、データ量をサポートするためにデータを複製したり、同じデータを異なる表現で提供したりすることができます。アド

ホック、探求的、分析的なユースケースにはファイルベースのエンドポイントを、より複雑なクエリに対応するために関係ベースのエンドポイントを、大量の集計を必要とするデータコンシューマにはカラムナデータベースのエンドポイントを、より高速で半構造化データ形式（JSON）を必要とするデータコンシューマにはドキュメントベースのエンドポイントを提供することができます。グラフデータベースを使用して、データプロダクトをノードと関係でモデル化することもできます。

[†19] エンタープライズデータウェアハウスをデータ流通に使用する場合、データのさまざまな形式や次元を簡単には利用できないことがよくあります。エンタープライズデータウェアハウスは、一般的に単一のテクノロジー（RDBMSデータベースエンジン）で設計されているため、さまざまなバリエーションを提供することができないのです。

さまざまなエンドポイントや出力ポートを提供する場合は、すべてを同じデータガバナンスのもとで管理する必要があります。同じ提供ドメインが説明責任を負います。データ再利用と読み出し最適化の原則は、データプロダクトのすべてのバリエーションにも適用されます。さらに、コンテキストと意味論は同じであることが要求されます。例えば、「顧客」や「顧客名」という概念は、提供されるすべてのデータで一貫していなければなりません。同じデータプロバイダーに関わるすべてのデータプロダクトに対して、同じユビキタス言語が使われなければなりません。7章では、すべてのエンドポイントで意味的一貫性がどのように確保されるのかなど、そのほかの統合パターンについて説明します。

4.5.5　データ提供サービス

データプロダクトアーキテクチャとユーザーやアプリケーションのやり取りを詳しく見ていくと、一般的には、多種多様なデータアクセスパターンが存在することがわかります。これらのパターンには、アドホッククエリ、ダイレクトレポーティング、セマンティックレイヤーやキューブの構築、ほかのアプリケーションへのODBC（Open Database Connectivity）の提供、ETLツールによる処理、データラングリング、データサイエンス処理などが含まれます。これらをサポートするためには、データプロダクトアーキテクチャに十分な性能とセキュリティ機能を持たせることが不可欠です。

データを消費するすべてのサービスにおいて、データへのアクセスとライフサイクル管理に一貫性を持たせたいはずです。これがデータ契約です。8章で説明します。

クラウドベースのオブジェクトストレージなどの分散ファイルシステムは、大量のデータを保存する場合にはコストパフォーマンスに優れていますが、一般的にアドホックでインタラクティブなクエリには十分に高速ではありません。多くのベンダーがこの問題を認識し、SQLクエリエンジンや軽量仮想化レイヤーを提供しています。データプロダクトに対してクエリを直接実行できるようにすることで、データを複製する必要がないため、アーキテクチャが安価になるという利点があります。また、これらのツールはデータへのきめ細かなアクセスを提供し、運用データストアは必要なくなります。データプロダクトアーキテクチャにあるデータは、運用レポートやアドホック分析の対象となります。データ仮想化オプションは良いソリューションですが、単純なレポート作成にしか使えません。これでは不十分な場合は、データの複製などの代替案を検討してください。

4.5.6　ファイル操作サービス

データの複製とデータの消費について説明してきましたが、ドメインによっては、入力にファイル（例えばCSVファイル）しか使用できないユースケースやアプリケーションがあります。このような場合、コンシューマが見てはならない機密データを自動的にマスクしたり除去したりするようなファイル操作サービスを構築することを検討してください。このサービスは、ほかの消費エンドポイントと同じように、中央セキュリティモデルに適合させ、データ消費をすべてデータ契約（「8.1.5　データ契約」を参照）に適合させなければなりません。また、フィルタは自動的に適用され、メタデータの分類と属性をベースに適用されます。これについては8章で詳しく説明します。

4.5.7　匿名化サービス

データ探索、データサイエンス、機械学習、第三者とのデータ共有のためにデータプロダクトを直接使用する場合、コンシューマから機密データを保護することが大切です。トークン化、ハッシュ化、スクランブル、暗号化などのセキュリティ手法を使用することで、ルールや分類ベースでデータを保護することができます。データプロダクトアーキテクチャをどのように設計するかによって、以下のような方法を適用することができます。

- データ消費実行時にデータを保護します。この手法では、保存されているデータに変更を加えることなく、データがクエリされたときにのみ機密データを保護します。例えばDatabricksでは、**ダイナミックビュー関数**（https://oreil.ly/HYm6T）と呼ばれる機能を使って、アクセス時にデータを隠したりマスクしたりします。
- 機密データがデータプロダクトアーキテクチャに取り込まれる前に、難読化したりマスクしたりします（トークン化など）。その後、消費段階で匿名化されたデータとの照合が行われます。
- プロジェクトごとにデータを複製して処理します。どのような分類、マスキング、フィルタリングルールを使用するかを定義することで、保護方法をカスタマイズできます。

データセキュリティはデータガバナンスに基づきます。この2つの領域については8章で詳しく説明します。

4.5.8　分散オーケストレーション

設計の検討と標準化に関しては、チームがデータパイプラインを実装、テスト、オーケストレーションする際のサポートも必要です。データパイプラインの構築と自動化は、データの抽出、準備、変換といった一連の工程を繰り返すプロセスです。チームがすべてのデータパイプラインと成果物の品質をテストおよび監視できるようにし、配備プロセスでのコード変更をサポートする必要があります。このエンドツーエンドの配信手法は、多くの自動化、技術的なプラクティス、ワークフロー、アーキテクチャパターン、ツールを含むという点で、DataOpsによく似ています。

ツールやベストプラクティスを標準化する際、筆者は、中央チームやプラットフォームチームを組織することを提案します。このチームは、ほかのチームをサポートし、スケジューリング、メタデータのキュレーション、メトリクスのログ記録などの機能を提供します。このチームは、エンドツーエンドの監視も担当すべきです。また、パイプラインが長時間にわたって停止している場合

も、このチームが対処します。例えば、依存関係のある複数のソースを組み合わせるときに、コンシューマに対して特別な処理フレームワークを構築し、さまざまなデータプロバイダーからの複雑な依存関係や時間間隔の違いを抽象化してもかまいません。

4.5.9　インテリジェントな消費サービス

このようなさまざまな機能をベースに、自動プロファイリング、フィルタリング、スキーマの整合、結合など、データ消費時にインテリジェントなデータ変換を行うレイヤー（ファブリック）によって、データプロダクトアーキテクチャを拡張することもできます。データコンシューマは必要なデータを指定し、希望する形式にモデリングされたデータを受け取ることができます。このような種類のインテリジェントサービスは、一般的に、意味レイヤーを配備し、複雑なロジックを隠し、使いやすい表現をビジネスユーザーに提示するためにも使われます。このサービスは、データガバナンスに特化したほかの多くのサービスを含みます。また、メタデータを多用します。将来性を見据えた場合、セマンティックナレッジグラフや機械学習といった今後のテクノロジーをファブリックに活用するのが良いでしょう。このトレンドは比較的新しいものですが、9章ではこのアプローチを実装するための準備として、いくつかの提案を行います。

4.5.10　直接利用に関する考察

設計において難しいのは、データプロダクトを消費ドメインの直接的なソースとして扱うべきか、または消費ドメインがデータプロダクトを抽出して複製する必要があるかということです。ここでは、永続的なデータ変換についての話をしています。永続的なデータ変換とは、データプロダクトアーキテクチャの外にある新たな場所（データレイクやデータベース）にデータが書き込まれ、保存されるプロセスを指します。コントロールされていないデータ複製は避けることが望ましいのですが、消費アプリケーションの近くや内部で複製が利用できる方が良い場合もあります。以下に例を挙げます。

- データプロダクトを使用して新しいデータを作成する場合、論理的にはこのデータは新しい場所に保存しなければならず、別のオーナーが必要になります。
- 遅延を減らしたいのであれば、データを複製して、消費するアプリケーションに近いローカルで利用できるようにしたほうが良いでしょう。6章で

は、完全にコントロールされた方法で、分散マテリアライズドビューを作成する方法を紹介します。

- 直接かつリアルタイムのデータ変換は、ユーザーエクスペリエンスを損なうほど負荷がかかる場合があります。例えば、データプロダクトに対するクエリに時間がかかりすぎて、ユーザーがイライラすることがあります。データの抽出、再構築、前処理を行うか、より高速なデータベースエンジンを使用することで、このような不満を和らげることができます。
- 消費アプリケーションの重要性が高く、データプロダクトアーキテクチャが動かなくなったときでもアプリケーションが動作できるようにするために、データを複製することが合理的な場合もあります。

上記のような場合に、データプロダクトごとにどのアプローチを採用するかを決める際には、エンタープライズアーキテクチャでコントロールするようにしてください。複製されたデータはクレンジングがしにくく、データがどのように使用されているかについてのインサイトを得ることが難しくなる場合があることに気をつけてください。もしデータが制限や管理されることなく複製できるのであれば、さらにデータが散らばってしまうことにもなりかねません。GDPRやCCPAのような法規制の遵守も難しくなります。このような問題への対応についても、11章で詳しく説明します。

4.6　始めるにあたって

データオーナーシップやデータプロダクトの開発に移行することは、多くのチームにとってカルチャー上の課題や技術的な課題に直面することになります。例えば、すべてのチームが必要な知識を持ち、データの価値を理解し、データに投資する意思を持っているわけではありません。データプロダクト開発への移行を自然な形で進めたい場合は、以下のベストプラクティスを検討してください。

- 小さく始めましょう。そして、データプロダクト開発という考え方を理解しているチームが存在するドメインから始めましょう。まずは成果を示し、その後規模を拡大しましょう。
- 組織におけるデータプロダクトの意味について、一貫した定義を作成します。この定義を、メタデータ管理に使用されているメタモデルと整合させます。
- ガバナンスは重要ですが、ゆっくりやりましょう。生産性を高めるために微妙なバランスを保ちましょう。オーナーシップ、メタデータ、相互運用

性の標準など、基盤となる要素に注力しましょう。できるだけ業界標準やベストプラクティスに合わせましょう。

- 最初のデータプロダクトは、他ドメインのお手本とならなければなりません。品質と設計基準を定め、データセットの設計にこだわりましょう。リソース指向APIを設計するのは、品質の高いデータプロダクトを構築する上で重要な第一歩です。
- 一度にすべてのユースケースをサポートするプラットフォームを構築しようとしてはいけません。最初の段階では、一般的に、プラットフォームは1種類のデータ取り込みと消費しかサポートしません。例えば、消費形式としては、バッチでのデータ移動やParquetファイルなどに限定します。
- 中央集権を許容してください。その後、非中央集権的に移行するようにしてください。最初にデータプロダクトを配備したときに、組織構造をすぐさま崩壊させる必要はありません。しばらくは中央集権を維持したまま、エンジニアをドメインに所属させたり、エンジニアを内容領域専門家（Subject Matter Expert）と一緒に働かせるためにソフト的な調整を行ったりします。最初のうちは、中央チームのサポートを受けながら、ドメイン固有のデータセットを作成してもかまいません。

始めたばかりの頃は、ゆっくりやっていきましょう。 学んでそれを繰り返し、次の段階に進みましょう。中央集権的なものと非中央集権的なものは、一長一短ではなく、微妙な違いであり、定期的に評価することで、常に較正していくものなのです。

4.7　まとめ

データプロダクトは、ドメインやアプリケーション間の依存関係の管理方法を大きく変えるものです。機能の関心事やコンテキスト境界の境界を越えるときは、チームは統一的なデータ流通方法に従うべきです。つまり、データをデータプロダクトとしてほかのチームに提供するという方法です。このような方法は、共有される依存関係が少ないため、チーム間の調整が少なくて済みます。

この章の冒頭で、「プロダクトとしてのデータ（data as a product）」と「データプロダクト」という概念を明確に区別しました。おさらいすると、「プロダクトとしてのデータ」とは、データを大規模に管理する際に用いる考え方です。信頼関係を構築し、ガバナンスを確立し、プロダクト思考を導入する、ということです。つまり、一番大切なデータを大事に扱い、原則を実践するということです。これは、ドメインチームが持つべきマインドセットです。チーム

はデータオーナーシップを持ち、データ品質の問題をソースで解決し、多くのユーザーにとって有用なデータを提供しなければなりません。中央チームがすべての変更を吸い上げ、データを統合するのではなく、ドメインチームは物理データモデルをドメイン境界内にカプセル化すべきです。これは、生データをとりあえず放り投げて、データ品質やモデル化の問題をコンシューマに解決させるのとは正反対なやり方です。ドメインチームは、最高のユーザーエクスペリエンスを提供できるように、データをモデル化し、流通する必要があります。

データプロダクトは、読者自身がその概念を明確にする必要があります。まず、組織内でデータプロダクト管理について議論する際に、ものごとを表現する方法を標準化することから始めてください。というのも、使われている言葉や定義に差異があるかもしれないからです。物理的な表現や、その基盤となる技術アーキテクチャに重点を置くのではなく、論理的な構成要素としてデータプロダクトを定義するようにしてください。どこかの時点で組織の意味モデルを導入し、データプロダクトをグラフィカルに表現し、そこからドメイン、データオーナー、ソースアプリケーション、物理的な表現との関係を描きます。これは少し概念的に聞こえるかもしれませんが、このようにデータプロダクトを管理することのメリットは、9章を読めば明らかになるでしょう。

データプロダクトには気を付けなければならないことがあります。マイクロサービスアーキテクチャと同じように、データが細かすぎたりバラバラだったり、何百もの異なる場所からデータを引っ張ってこなければならないという問題に悩まされる可能性が高いのです。これらの問題を解決するには、適切な標準を確立し、中央組織を設置し、すべてのデータプロダクトの作成処理を監督します。データ交換のための相互運用性標準も設定する必要があります。データの変更やモデル化、提供方法についても調整が必要です。残念ながら、このようなことを行うのは簡単ではありません。これまで学んできたように、データプロダクトの開発はデータウェアハウスと密接に関係しており、データウェアハウスはデータモデル（データの一貫性、読み出し可能性、発見可能性、アドレス可能性、信頼性、セキュア化）など多くの分野から構成されています。これらの中には、この章で説明した履歴の作成のように複雑なものもあります。

データプロダクトとdata as a productの考え方は、**データ提供**に関する考え方です。**データ消費**はその次の段階の話です。11章で学ぶように、一部の作業や領域は重なるかもしれませんが、データを提供することとデータを消費するこ

との関心事は同じではありません。データをプロダクトとして管理し、データプロダクトをドメインインタフェースとして提供することで、依存関係をなくし、対照的な関心事を持つアプリケーションをうまく分離することができます。依存関係が少なくなるということは、チーム間の調整が少なくてすむということなので、これは重要なことです。

データプロダクトアーキテクチャは、組織のニーズに基づいて協力して設計する必要があります。ドメインチームに汎用的なブループリントを提示しましょう。ほかのチームがプラットフォームチームに貢献したり、一時的に参加したりできるようにします。例えば、新しい消費パターンが必要な場合は、再利用可能なパターンやコンポーネントとしてそれを継続的に提供します。データの取り込みと導入も同様です。最新のデータプラットフォームをスケールアップして構築する場合、多くのセルフサービスアプリケーションコンポーネントを追加する必要があります。これらには、中央管理されたETLフレームワーク、変更データキャプチャサービス、リアルタイムインジェストツールなどが含まれるでしょう。

最後に重要な注意点があります。本章で述べたように、データプロダクトはデータを利用可能にするためのものです。データをどのように利用し、本当の価値に変えるかについてはまだ説明していません。これはデータプロダクトの作成とは異なるものです。11章でまた説明します。

次の章では、APIアーキテクチャについて見ていきます。APIアーキテクチャは、サービス通信や、リアルタイムで低遅延なユースケース用に少量のデータを流通させる際に使用されます。